Implementation Project Graph Isomorphism

Authors: Jelle Boon, Wouter Fokkema, Ruben Hoeksma, Bram Kanger, and Daan van Kats

Faculty EEMCS
University of Twente

Version: January 18, 2026

Contents

1 Introduction **1**

1.1 Similarity, symmetry and isomorphisms 1

1.2 Problem Definition 2

1.3 Relation Diagram 3

2 Colour Refinement **5**

2.1 Intuition 5

2.2 Colouring of a graph 6

2.3 Refining a Colouring 9

2.4 Stable and (Un)balanced Colourings 15

2.5 Partitions 18

2.6 Relation to the #Aut problem 21

3 Branching **23**

3.1 Intuition 23

3.2 Example 26

3.3 #Aut Problem 32

3.4 Implementation Details 33

3.5 Improvements 36

4 Fast Colour Refinement **37**

4.1 Intuition 37

4.2 Minimising Deterministic Finite Automaton 37

4.3 Faster Minimisation Algorithm 42
4.4 Application on the GI/#Aut Problems 50

5 Automorphism Groups 53
5.1 Introduction 53
5.2 Generating Sets 57
5.3 Polynomial Size Generating Sets 59
5.4 Pruning the Branching Tree 61
5.5 Order Computation and Membership Testing 66
5.6 Implementation Details 72

6 Graph Structures and Preprocessing 75
6.1 Preprocessing 75
6.2 Trees 76
6.3 Twins 78
6.4 Components 79
6.5 Recursive Preprocessing 79
6.6 Other structures 80

Bibliography 83

Index 84

Chapter 1

Introduction

1.1 Similarity, symmetry and isomorphisms

This reader provides the mathematical background that is necessary for the implementation project graph isomorphism. The main goal of this reader is to provide explanations for why the suggested algorithms are correct, how they should be implemented (at a high level), and what further extensions can be considered.

Symmetry is a recurring theme in many branches of (discrete) mathematics. An example from optimisation is the detection and elimination of symmetries in mathematical programs, which is key to create efficient software. Given the description of two objects, investigating their symmetry is asking the seemingly innocent question if they are actually different or not. In more mathematical terms, we call two given objects *isomorphic* if they describe "essentially" one and the same thing. Loosely speaking, that means that for two such given objects, there exists a bijective mapping that "preserves structure". We will start by formalising what this means exactly for finite graphs.

The focus of the project is on practical implementations of algorithms for the notorious **graph isomorphism** problem, a problem that is still open with respect to its computational complexity[1]. Up to now it has not been proven to be in P and neither to be NP-complete. Relatively recently, there was a major breakthrough result, that puts the problem closer to P than being NP-hard [2].

Algorithms that decide if two given input graphs are isomorphic or not, and algorithms that detect the (number of) symmetries of a given graph, are useful in many practical applications: Mathematical Optimisation, Computational Biology, Fingerprint Recognition, etc.

[1] You will learn about the complexity classes later during the module.

1.2 Problem Definition

We briefly define some necessary notions related to graph isomorphisms and automorphisms. Many of these notions are likely known, but we include them for completeness. We consider simple, undirected, finite graphs. We usually define a graph by a tuple (V, E). Where V is a set of *vertices*, and E is a set of *edges* (each edge is an unordered pairs of vertices). By $V(G)$ and $E(G)$ we denote the sets of vertices and edges of the graph G, respectively. We will define what an isomorphism is. Before we can do so, we need to define what a *bijection* is.

Definition 1.1 (Bijection)**.** A mapping $f : V \to W$ is *bijective* if an only if f is both onto and one-to-one. We call a bijective mapping a *bijection*.

Here, one-to-one and onto are defined as follows.

Definition 1.2 (One-to-one)**.** A function $f : V \to W$ is one-to-one (injective) if $\forall\, a, b \in V$, if $f(a) = f(b)$, then $a = b$.

Definition 1.3 (Onto)**.** A function $f : V \to W$ is onto (surjective) if $\forall y \in W$, $\exists\, x \in V$, s.t. $f(x) = y$.

As said in Section 1.1, an isomorphism between two graphs is a bijection from the vertices both graphs. Intuitively, we call a bijection between the vertex sets of two graphs an isomorphism if it preserves the structure of the graph; edges between vertices translate to edges between the images of those vertices.

Definition 1.4 (Isomorphism)**.** An *isomorphism* between two graphs G and H is a bijection $f : V(G) \to V(H)$ such that for all $u, v \in V(G)$:

$$\{u, v\} \in E(G) \quad \Longleftrightarrow \quad \{f(u), f(v)\} \in E(H)\,. \tag{1.1}$$

Two graphs G and H are called *isomorphic* if there exists an isomorphism between them. Note that, since an isomorphism is a bijection, we know that two graphs can only be isomorphic if they have an equal number of vertices.

Definition 1.5 (The Graph Isomorphism (GI) Problem)**.** The *Graph Isomorphism problem* asks whether there exists a isomorphism between two graphs G and H.

Notice that the GI-problem is a *decision problem*, the answer to the question is either *yes* or *no*. The goal of this project is to develop an algorithm that efficiently answer this question.

The second part of this project is to determine how many automorphisms a graph G has, also know as the *#Aut problem*. Of course we must first look at what a automorphism is.

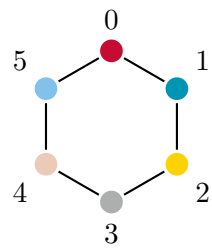


Figure 1.1: A cyclic graph (C_6) with 6 distinctly coloured vertices.

Definition 1.6 (Automorphism)**.** An automorphism of a graph $G = (V, E)$ is a permutation σ of the vertex set V (a bijection from V to V), such that the pair of vertices (u, v) form an edge if and only if the pair $(\sigma(u), \sigma(v))$ also form an edge. That is, it is a isomorphism from G to itself.

Definition 1.7 (The Automorphism Counting (#Aut) Problem)**.** The *Automorphism Counting Problem* asks, given a graph G, how many automorphisms G has.

Notice that the #Aut problem is a *counting problem* and the solution is a number. To give an illustrative example, consider the graph from Figure 1.1. You can try and see for yourself how many automorphisms you can come up with. All cyclic permutations, also called rotations, are automorphisms. More precisely, if $V = \{0, \ldots, 5\}$ and $E = \{\{0, 1\}, \{1, 2\}, \ldots, \{4, 5\}, \{5, 0\}\}$, the mapping $i \mapsto i + k \mod 6$ is an automorphism for all $k \in \mathbb{Z}$. In fact, the automorphisms of any graph G form an algebraic *group* under composition. This last fact turns out to be incredibly useful for solving the #Aut problem for large graphs in a reasonable time. Chapter 5 discusses this in detail.

1.3 Relation Diagram

The algorithms and methods that combine to solve the Graph Isomorphism and Automorphism Counting problems relate to and depend on each other in different ways. Knowing and understanding these relations will help you plan and execute your project. The relations and dependencies are visualised in Figure 1.2.

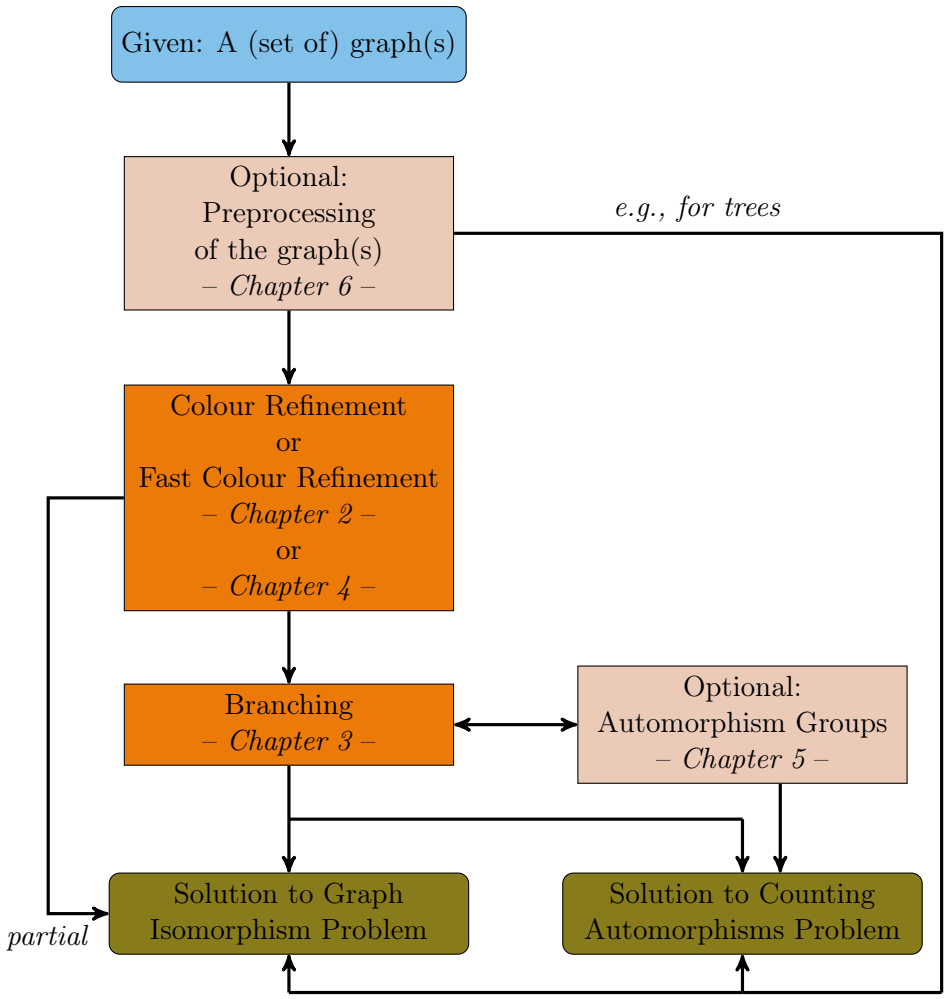


Figure 1.2: An overview of the different elements to a working algorithm.

Chapter 2

Colour Refinement

In Chapter 1, we introduced the Graph Isomorphism Problem and the Automorphism Counting Problem. Throughout the project and, therefore, in this reader, we look for methods which solve these problems for any pair of graphs.

In this Chapter, we introduce a first concept that helps us tackle this problem, a procedure called *colour refinement*. Colour refinement (often) helps us distinguish two graphs from each other more easily. In the following chapters we further extend this procedure to ensure that it distinguishes all non-isomorphic graphs and we investigate ways to speed it up. But for now, we stick to this fairly simple, yet effective, method.

We start in Section 2.1 by repeating the basics of the mathematical definition of isomorphism: the whole basis of our project. In Section 2.2, we will formalise what it means to create a *colouring* of a graph, or more precisely, of its vertices. In Section 2.3, the method that refines these colourings is introduced. We will also see how refining colourings give us more information about the structure of the graph. In Section 2.4, we will be describing the stop condition for the refining procedure and see what that means mathematically. This will help us transforming the procedure to an algorithm. Then lastly, in Section 2.5, we will look into the question whether our colour refinement algorithm will allow us to to determine whether graphs are isomorphic.

2.1 Intuition

We first explore the GI-problem through an example consisting of three graphs.

Example 2.1. *Consider the three graphs in Figure 2.1. Which of them are (pairwise) isomorphic? How would you approach this question?*

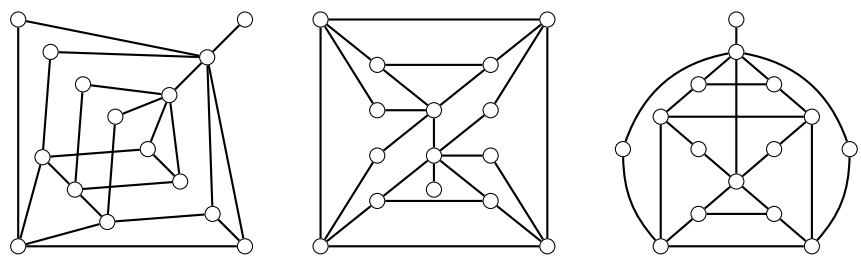


Figure 2.1: Three example graphs.

2.1.1 First idea: Enumeration

A intuitive idea, and one you may came up with yourself, is to solve the GI Problem for our example by checking all possible bijections for every graph pair. For every such bijection, one then needs to check whether the bijection satisfies Definition 1.4. A fast calculation immediately teaches us that there are, for two graphs with n vertices per graph, a total of $n!$ bijections from one vertex set to the other. For the relatively small graphs in Example 2.1, where $n = 15$, this already means that we may need to check $15! = 1.307.674.368.000$ bijections. This forms a bit of a problem for this approach, as, even if you can verify a thousand bijections per second, it will take you more than 40 years of computations, before you can definitively answer the question for a non-isomorphic graph pair[1].

We need to work smarter, since working harder is definitely infeasible for large graphs.

2.2 Colouring of a graph

In this section, we explore the concept of a *colouring* of a graph. Before we start with the mathematical definition of a colouring, let us ask what does it mean for a graph to have a colouring?

A (vertex) colouring is an abstract concept that gives every vertex of a graph a certain "colour". To not restrict ourselves to the number of colour names that the average human knows, we will identify "colours" by a number and instead of saying that a vertex has colour grey, we say, e.g., it has colour 1.

Definition 2.2 (Colouring)**.** Let G be a graph. A *k-vertex-colouring*, usually

[1] As an exercise, think about the difference for this algorithm between an isomorphic and a non-isomorphic graph pair

shortened to *k-colouring*[2], of G is a function $\alpha : V(G) \to \{1, \ldots, k\}$. Here, $\alpha(v)$ is called the *colour* of vertex v.

We also define a k-colouring of a set of graphs $\mathcal{G} = \{G_1, \ldots, G_\ell\}$ as a function $\alpha : \bigcup_{i=1}^{\ell} V(G_i) \to \{1, \ldots, k\}$.

For a set of graphs $\mathcal{G} = \{G_1, \ldots, G_\ell\}$, we denote $V(\mathcal{G}) = \bigcup_{i=1}^{\ell} V(G_i)$.

Often, when we don't know exactly the number of colours used or it is clear from context, we simply refer to a *colouring*, implying $\alpha : V(\mathcal{G}) \to \mathbb{N}_0$.

Let us first consider the following very simple colouring: the *uniform colouring*. This is a 1-colouring where every vertex of a (set of) graph(s) has colour 1 (see Figure 2.2),

$$\alpha_{uniform}(v) = 1, \quad \forall\, v \in V(\mathcal{G})\,.$$

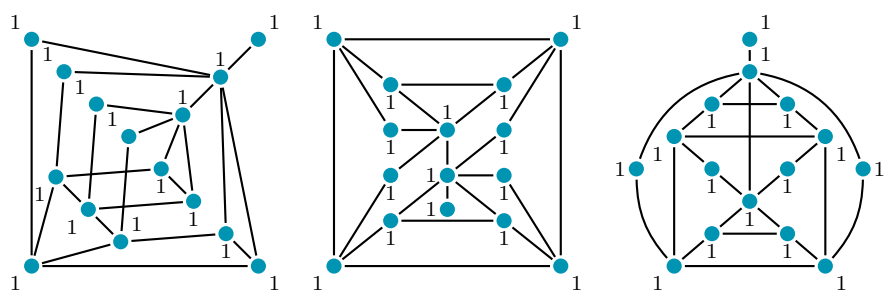


Figure 2.2: The uniform colouring of the graphs of Example 2.1

In this course we often refer to colour-preserving bijections between the vertices of two graphs.

Definition 2.3 (Colour-preserving bijection)**.** Let α be a k-vertex colouring of the set of graphs $\mathcal{G} = \{G, H\}$. The bijection $f : V(G) \to V(H)$ is said to be *colour-preserving* if for every $v \in V(G)$: $\alpha(v) = \alpha(f(v))$.

2.2.1 Colouring based on degree

Intuitively, two graphs are isomorphic if they are structurally identical. In the rest of this chapter, we aim to capture some of that structure by smartly colouring the vertices of the graphs. The neighbourhood of a vertex v consists of all vertices that share an edge with v.

[2]Note that also, e.g., edge-colourings exist, yet when we speak of a k-colouring or a colouring of a graph this usually refers to vertex colouring.

Definition 2.4 (Neighbourhood)**.** Let G be a graph and let $v \in V(G)$ be one of its vertices. The *neighbourhood*, $N(v)$, of v is the set of vertices that share an edge with v. That is,

$$N(v) = \{u \in V(G) \mid (u, v) \in E(G)\}\,.$$

Let us continue with the following observation leading from Definition 1.4:

Observation 2.5. *If $f : V(G) \to V(H)$ is an isomorphism, then for every $v \in V(G)$, we have that f yields a bijection from $N(v)$ to $N(f(v))$. That is,*

$$f(u) \in N(f(v)) \quad \Leftrightarrow \quad u \in N(v)\,.$$

Proof. Assume that $u \in N(v)$. Then, we know that $\{u, v\} \in E(G)$. Therefore, we know that $\{f(u), f(v)\} \in E(H)$, which, by definition, implies that $f(u) \in N(f(v))$. The proof for the opposite implication is analogous. □

Since the degree of a vertex simply counts its neighbours, we obtain the following corollary.

Corollary 2.6. *If f is an isomorphism from G to H, then for every $v \in V(G)$, we have*

$$d(v) = d(f(v))\,.$$

In other words, an isomorphism is always "degree-preserving".

We can capture the degrees of the vertices with the *degree colouring* of a (set of) graph(s), given by

$$\alpha_{degree}(v) := d(v)\,. \tag{2.1}$$

Figure 2.3 shows a degree colouring of the graphs of Example 2.1.

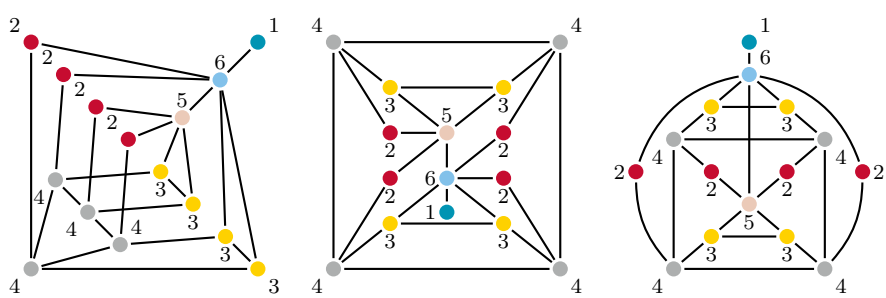


Figure 2.3: Degree-colouring of Example 2.1

By Corollary 2.6, we know that an isomorphism from one graph to any other graph must be colour-preserving for the degree colouring. Any 'blue' vertex must be mapped to a 'blue' vertex in the other graph. This means that we have to check only those bijections that satisfy this condition. In Example 2.1, there are only $4! \cdot 4! \cdot 4! = 138234$ bijections left that need too be checked. This is already a big improvement from the total enumeration from Section 2.1.1. This is, however, not good enough if we want to be able to compare any (much larger) graph. In the next sections, we discuss a method that iteratively improves on a colouring such that the obtained colourings still satisfy the invariant that any isomorphism is colour-preserving.

2.3 Refining a Colouring

In this section, we continue to build on the idea of colouring graphs to recognise structures. At the end of the previous section, we found that it is already much more efficient to only check bijections that preserve degree, instead of exhaustively checking all bijections to find (or not find) an isomorphism. We extend this idea by creating another, more intricate, colouring of our graphs that is also preserves colour under isomorphisms. We obtain this colouring through the *colour refinement* algorithm. Intuitively, refinement uses the current colouring to create a new colouring with the desired properties.

Before we introduce the refinement procedure, we first define some necessary concepts related to colourings.

Definition 2.7. Let α be a colouring of a set of graphs $\mathcal{G}$. For every colour c from α, we define

$$\alpha^c := \{v \in V(\mathcal{G}) \mid \alpha(v) = c\}$$

to be the set that contains all vertices with colour c.

As a simple example, consider the uniform colouring. Since this colouring has only one colour, we have only one α^c, which consists of all the vertices of the set of graphs.

Instead of only comparing the colour of a single vertex, we can also compare the colours of a set of vertices. In particular, in the context of isomorphisms, it is useful to consider the colours of the neighbours of a vertex. We want to distinguish vertices that have *differently-coloured* or *identically-coloured* neighbourhoods.

Definition 2.8 (Differently/Identically-Coloured Neighbourhoods)**.** Let α be a colouring of the set of graphs $\mathcal{G}$. Two vertices u, v are said to have *differently-coloured neighbourhoods* if there exists a colour c, such that

$$|N(u) \cap \alpha^c| \neq |N(v) \cap \alpha^c| \,.$$

That is, the number of c-coloured neighbours of u is different from the number of c-coloured neighbours of v. If u and v do not have differently-coloured neighbourhoods, we say they have *identically-coloured neighbourhoods.*

The following observation connects colour-preserving isomorphisms to Definition 2.8.

Observation 2.9. *Let α be a colouring of the set of graphs $\mathcal{G} = \{G, H\}$ and let f be a colour-preserving isomorphism from G to H. Then for every $v \in V(G)$, we have that v and $f(v)$ have identically coloured neighbourhoods.*

Proof. Follows from Definition 2.3 and Observation 2.5. □

Observation 2.9 is gives us a crucial tool to find isomorphisms. It provides a condition which must hold for two isomorphic graphs. The only catch is that instead of for an "isomorphism", the statement holds for any "colour-preserving isomorphism". Thus, we look for a way to colour our graphs, such that *any* isomorphism is colour-preserving. Recall that we have seen two examples of such colourings so far, the uniform colouring and the degree colouring.

We are now ready to introduce the main topic of this chapter, the colour refinement procedure. It describes how to refine a given colouring to a new one.

Definition 2.10 (Colour Refinement)**.** Let α_i be a colouring of the set of graphs $\mathcal{G}$. The colouring α_{i+1} is a refinement of α_i, such that for every two vertices $u, v \in V(\mathcal{G})$, $\alpha_{i+1}(u) = \alpha_{i+1}(v)$ if and only if

(i) $\alpha_i(u) = \alpha_i(v)$, that is, u and v had the same colour in α_i, and

(ii) u and v have identically coloured neighbourhoods in α_i.

Remark. *For vertices, 'having the same colour' in a colouring α_i is an* equivalence relation*, which we denote by $v \equiv_\alpha u$ if v and u have the same colour in α* [3]*.*

Using the equivalence relation notation, colour refinement considers equivalence classes of the colouring α_i ((i), translates to $u \equiv_{\alpha_i} v$) and recolours them based on the colouring of their neighbourhoods.

The use of colour refinement becomes apparent when we realise that each refinement maintains the following invariant.

Lemma 2.11. *If an isomorphism $f : V(G) \to V(H)$ is colour-preserving for colouring $\alpha_i : V(G) \cup V(H) \to \mathbb{N}_0$, then f is also colour-preserving for α_{i+1}, the result of the colour refinement procedure on α_i.*

[3] An equivalence relation is a binary relation $\simeq$ that is reflexive ($v \simeq v$ for all v), is symmetric ($(v \simeq u) \Leftrightarrow (u \simeq v)$ for all v and u), and is transitive ($(v \simeq u) \wedge (u \simeq w) \Rightarrow (w \simeq v)$ for all u, v and w).

Proof. Assume that f is colour-preserving for α_i. That is, $v \equiv_{\alpha_i} f(v)$ for all $v \in V(G)$. By Observation 2.5, we know that f maps the neighbours of v to the neighbours of $f(v)$. Since f is colour-preserving for α_i, we know that v and $f(v)$ have identically-coloured neighbourhoods under α_i. Now, by Definition 2.10, since v and $f(v)$ have the same colour under α_i and have identically-coloured neighbourhoods under α_i, we have that $v \equiv_{\alpha_{i+1}} f(v)$. □

2.3.1 Colour refinement - A Full Example

We apply Colour Refinement on Example 2.1.

Refinement 0 (Initial colouring) The procedure does not specify with which colouring we start. However, we do have an initial condition, namely that all isomorphisms need to be colour-preserving for our initial colouring. We therefore start with the uniform colouring α_{uniform} of the set of the three graphs [4]. See Figure 2.4.

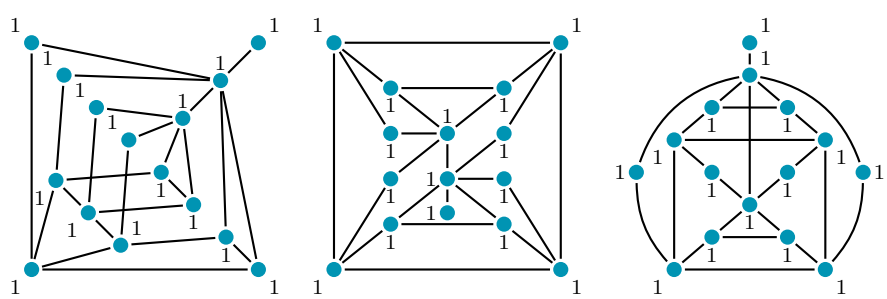


Figure 2.4: Initial (uniform) colouring.

Refinement 1 For the first refinement, we compare the neighbourhoods of all the vertices of the three graphs. The three graphs have six pairwise non-identically-coloured neighbourhoods. These correspond to vertices with distinct degrees. See Table 2.1. Figure 2.5 shows the colouring after the first refinement, which indeed is exactly the degree colouring.

[4] Alternatively, we could start with the degree colouring, since both of them satisfy this condition.

Colour	Previous Colour	Colours in neighbourhood in Figure 2.4
1	1	1× ‘1’
2	1	2× ‘1’
3	1	3× ‘1’
4	1	4× ‘1’
5	1	5× ‘1’
6	1	6× ‘1’

Table 2.1: Colours during Refinement 1 and their “meaning”.

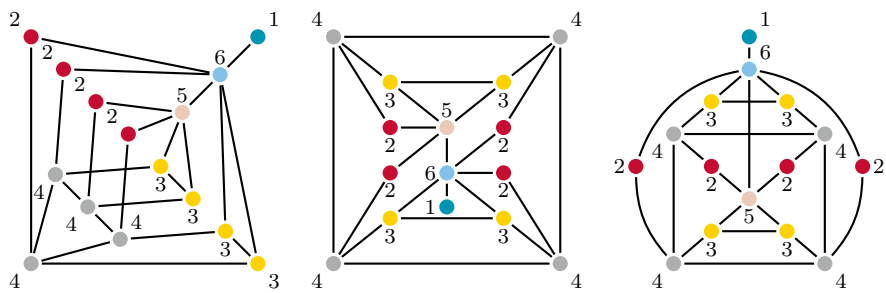


Figure 2.5: Coloured graphs after Refinement 1.

Refinement 2 In the second refinement step, we distinguish the two vertices with colour 2 that have a vertex with colour 5 as neighbour from the two vertices with colour 2 that have a vertex with colour 6 as neighbour. We do the same for the vertices with colour 3. This results in two more colours, which gives us eight colours in total. See Table 2.2 and Figure 2.6.

Colour	Previous Colour	Colours in neighbourhood in Figure 2.5
1	1	1× ‘6’
2	2	1× ‘4’; 1× ‘6’
7	2	1× ‘4’; 1× ‘5’
3	3	1× ‘3’; 1× ‘4’; 1× ‘6’
8	3	1× ‘3’; 1× ‘4’; 1× ‘5’
4	4	1× ‘2’; 1× ‘3’; 2× ‘4’
5	5	2× ‘2’; 2× ‘3’; 1× ‘6’
6	6	1× ‘1’; 2× ‘2’; 2× ‘3’; 1× ‘5’

Table 2.2: Colours during Refinement 2 and their “meaning”.

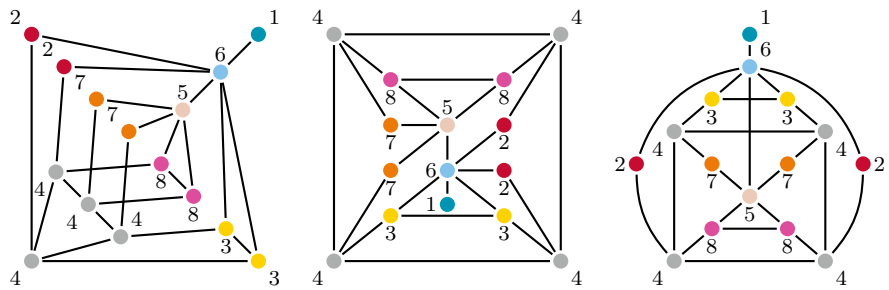


Figure 2.6: Coloured graphs after Refinement 2.

Refinement 3 We repeat the process on the colouring of Figure 2.6. This refinement results in three new colours: 9, 10 and 11. See Table 2.3 and Figure 2.7.

Colour	Previous Colour	Colours in neighbourhood in Figure 2.6
1	1	1× ‘6’
2	2	1× ‘4’; 1× ‘6’
3	3	1× ‘3’; 1× ‘4’; 1× ‘6’
4	4	1× ‘2’; 1× ‘8’; 2× ‘4’
9	4	1× ‘3’; 2× ‘4’; 1× ‘7’
10	4	2× ‘4’; 1× ‘7’; 1× ‘8’
11	4	1× ‘2’; 1× ‘3’; 2× ‘4’
5	5	1× ‘6’; 2× ‘7’; 2× ‘8’
6	6	1× ‘1’; 2× ‘2’; 2× ‘3’; 1× ‘5’
7	7	1× ‘4’; 1× ‘5’
8	8	1× ‘4’; 1× ‘5’; 1× ‘8’

Table 2.3: Colours during Refinement 3 and their “meaning”.

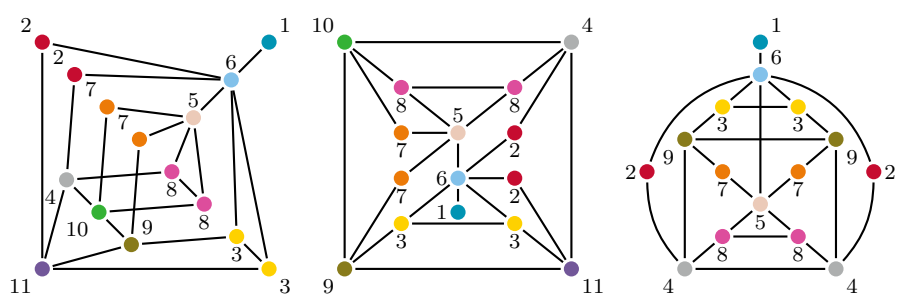


Figure 2.7: Coloured graph after Refinement 3

In Figure 2.7, we see that the graph on the right does not have a vertex with colour 10 (or on with colour 11), while the left and middle graph do contain this colour. This implies that no bijection from the right graph to one of the other two can be colour-preserving. Therefore the right graph cannot be isomorphic to the other two graphs and we continue only with the left and the middle graph.

Refinement 4 We now perform a last refinement, but only on the left two graphs. See Table 2.4 and Figure 2.8.

Colour	Previous Colour	Colours in neighbourhood in Figure 2.7
1	1	1× ‘6’
2	2	1× ‘4’; 1× ‘6’
12	2	1× ‘6’; 1× ‘12’
3	3	1× ‘3’; 1× ‘6’; 1× ‘9’
15	3	1× ‘3’; 1× ‘6’; 1× ‘11’
4	4	1× ‘2’; 1× ‘8’; 2× ‘4’
5	5	1× ‘6’; 2× ‘7’; 2× ‘8’
6	6	1× ‘1’; 2× ‘2’; 2× ‘3’; 1× ‘5’
7	7	1× ‘5’; 1× ‘9’
13	7	1× ‘5’; 1× ‘10’
8	8	1× ‘4’; 1× ‘5’; 1× ‘8’
14	8	1× ‘5’; 1× ‘8’; 1× ‘10’
9	4	1× ‘3’; 1× ‘7’; 1× ‘10’; 1× ‘11’
10	4	1× ‘4’; 1× ‘7’; 1× ‘8’; 1× ‘9’
11	4	1× ‘2’; 1× ‘3’; 2× ‘9’

Table 2.4: Colours during Refinement 4 and their “meaning”.

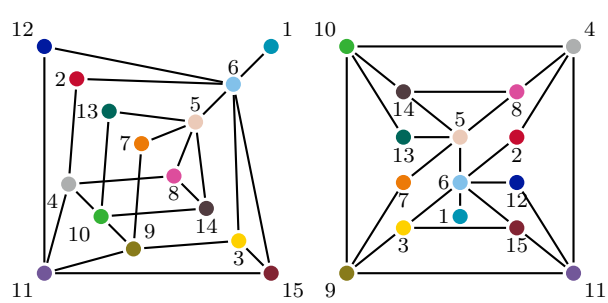


Figure 2.8: Coloured graph after Refinement 4

We see that both graphs in Figure 2.8 have exactly one of each vertex with colours

1 till 15, while they also have 15 vertices each. We give this type of colouring (where each vertex has a unique colour among the vertices of a single graph) a name since it plays an important role in the following sections.

Definition 2.12. A k-colouring α of a graph G is *discrete* if for all $c \in \{1, \ldots, k\}$: $|\alpha^c| = 1$.

Note that the exact colours (or numbers) used in a colouring do not matter. Thus, if we colour a graph G with $n = |V(G)|$ distinct colours, such that each colour is used, we say that the colouring is discrete.

If we look at the colouring of our example in Figure 2.8, we may already draw a conclusion about if the graphs are isomorphic. We know that any isomorphism is colour-preserving for this colouring, so the only possible potential isomorphism is the bijection that maps the vertex with colour c in the left graph to the vertex with colour c in the middle graph for all $c \in \{1, \ldots, 15\}$. Checking condition (1.1) for this bijection shows that the two graphs are indeed isomorphic.

In sections 2.4 and 2.5, we further discuss the colour refinement algorithm.

2.4 Stable and (Un)balanced Colourings

Initially, our goal of the colour refinement process was to reach a colouring such that every isomorphism is colour preserving. By Observation 2.9, we have a useful tool to check if two graphs are isomorphic. But when can we stop refining? Knowing when we can stop the process (set a stop condition), allows us to create an algorithm from Definition 2.10. Let us, once more, consider the colouring of Figure 2.8. If we refine this colouring once more, we notice that there is no change. Each vertex has its own colour, so no two vertices of the same colour can be distinguished by the colour of their neighbourhoods. An obvious stop condition is when the procedure does not change the colouring anymore. We call such a colouring a *stable colouring*.

Definition 2.13. A colouring α of graph G (or a set of graphs) is called *stable* if, for all $u, v \in V(G)$, we have $\alpha(u) = \alpha(v)$ implies that u and v have identically-coloured neighbourhoods.

Observation 2.14. *The colour refinement procedure ends in a stable colouring, regardless of the initial colouring.*

Proof. Consider the refinement procedure in Definition 2.10. If the iterative procedure terminates, then (i) implies (ii) and we have a stable colouring by definition. It remains to show that the procedure terminates. Consider again a refinement

during one iteration of the procedure. Any vertices that had distinct colours before the refinement, still have distinct colours after the refinement. Moreover, if the refinement did not terminate at this iteration (the resulting colouring is different from the one we started with), there is at least one colour that was split into at least two colours. Therefore the number of distinct colours strictly grows in each iteration until the procedure terminates. Since our graphs have a finite number of vertices, the procedure terminates in a finite number of iterations. □

Pseudocode for the colour refinement

With the stop condition from the previous paragraph, we define the following algorithm that takes any graphs and results in a stable colouring for which all isomorphisms between any pair of graphs are colour preserving.

Algorithm 1 Colour Refinement

Input: A set of graphs $\mathcal{G} = \{G_1, \ldots, G_\ell\}$.
Output: A stable colouring α of $\mathcal{G}$.

for all $v \in V(\mathcal{G})$ **do**
  $\alpha_0(v) := 1$
i := 0
while $\alpha_i \neq \alpha_{i-1}$ **do**
  i := i + 1
  Create a colouring α_i of $\mathcal{G}$ such that $\forall\, u, v \in V(\mathcal{G})$:
    $\alpha_i(u) = \alpha_i(v) \iff$
      • $\alpha_{i-1}(u) = \alpha_{i-1}(v)$, and
      • u and v have identically coloured neighbourhoods in α_{i-1}.
return α_i

Balanced and Unbalanced Colourings

We know now that our actual goal in the process of refinement is to reach a stable colouring. But what can we say afterwards about the Graph Isomorphism problem? In our Example in Section 2.3.1, the refinement of the left two graphs and the right graph resulted in colours that appeared in two graphs but not in the third. From this we concluded that two graphs are not isomorphic. Formally, we speak about balanced and unbalanced colourings.

Definition 2.15. Let α be a colouring of the set of graphs $\mathcal{G} = \{G, H\}$. α is called *balanced* if for every colour $c \in \{1, \ldots, k\}$, we have that

$$|\alpha^c \cap V(G)| = |\alpha^c \cap V(H)|\,.$$

Definition 2.16. Let α be a colouring of the set of graphs $\mathcal{G} = \{G, H\}$. α is called *unbalanced* if it is not balanced.

Definition 2.17. Let α be a k-colouring of the set of graphs $\mathcal{G} = \{G, H\}$. α is said to *define a bijection* f if for every colour $c \in \{1, \ldots, k\}$, we have that

$$|\alpha^c \cap V(G)| = |\alpha^c \cap V(H)| = 1\,.$$

In this case, the bijection $f : V(G) \to V(H)$ is chosen such that for all $v \in V(G)$, $\alpha(v) = \alpha(f(v))$.

Note that every colouring α that defines a bijection must be balanced, since Definition 2.17 only has the extra restriction that $|\alpha^c \cap V(H)| = 1$. Also, every colouring α that defines a bijection is discrete for each of the graphs G and H separately.

Now, we can make our conclusions with respect to the Graph Isomorphism problem.

Observation 2.18. *Let α be the stable colouring of the set of graphs $\mathcal{G} = \{G, H\}$ that is the result of the colour refinement algorithm (starting with the uniform colouring). If α defines a bijection $f : V(G) \to V(H)$, then f is the unique isomorphism from G to H.*

Proof. We will do our proof by induction.

1. *Base step*: Iteration 0: Any isomorphism g is colour preserving for the uniform colouring α_{uniform}.
2. *Induction step*: By Lemma 2.11, the isomorphism g remains colour preserving in each refinement step. Since α defines a bijection $f : V(G) \to V(H)$, it must be true that $f = g$.

□

If we find an unbalanced colouring through colour refinement, we can conclude that the graphs are not isomorphic.

Observation 2.19. *Let α be a stable colouring of the set of graphs $\mathcal{G} = \{G, H\}$ that is the result of the colour refinement algorithm (starting with the uniform colouring). If α is unbalanced, then there is no isomorphism from G to H.*

Proof. Since the colouring is unbalanced, there exists no colour preserving bijection from G to H. And, since al isomorphisms from G to H must be colour preserving, no such isomorphism can exist. □

Based on the colouring α that is the outcome of the colour refinement algorithm on the set of graphs $\mathcal{G} = \{G, H\}$, we can distinguish the following kinds of stable colourings:

- If α is unbalanced, we have solved the Graph Isomorphism Problem: G and H *are not* isomorphic.
- If α is balanced and defines a bijection, we have solved the Graph Isomorphism Problem: G and H *are* isomorphic.
- If α is balanced, but it does not define a bijection, we have not yet solved the Graph Isomorphism Problem. This case is the main topic of Chapter 3.

2.5 Partitions

A question that an attentive reader could ask right now is whether a stable colouring of a graph after colour refinement is (always) unique (if this is not the case, it makes our life difficult when we consider the #Aut problem).

The first answer is 'No'. The simple explanation is that we never specified *how* new colours are assigned to vertices. The colour refinement procedure in Definition 2.10 only tells us when vertices are given a distinct colour and when not, it does not specify which colour.

Let us illustrate this by looking back at the execution of colour refinement on the example of Section 2.3.1 again. More specifically, consider Table 2.2. During Refinement 2, we determined that the vertices which had the colours 2 and 3 in Figure 2.5 needed to be separated from each other in the new colouring. In our example, we split colour 2 into 2 and 7 and colour 3 into 3 and 8. Yet, instead splitting colour 2 into 2 and 8 and colour 3 into 3 and 7 would also satisfy the refinement procedure.

The actual answer to our uniqueness question is 'Yes', if we ignore the chosen colours and instead focus on the equivalence classes created. In that sense, the colouring resulting from colour refinement is unique. More precisely, the *partition* that results from colour refinement is unique.

A partition is a general mathematical concept in the context of sets (for our purpose, sets of vertices).

Definition 2.20 (Partition)**.** Let V be a set. A *partition* π of V is a set of non-empty subsets of V, such that every element of V appears in exactly one subset. Each such subset of V is called a *cell*.

Let u, v be two vertices in the set V and π be a partition of V. We write $u \equiv_\pi v$ to denote that u and v appear in the same cell of π.

Remark. *The binary relation $\equiv_\pi$ is an* equivalence relation.

The attentive reader can probably guess what partition of the vertices we relate to a colouring.

Definition 2.21. Let α be a k-colouring of the set of graphs $\mathcal{G} = \{G_1, \ldots, G_\ell\}$. We associate the following *partition of vertices into colour classes* to α:

$$\pi_\alpha = \{\alpha^c \text{ , s.t. } |\alpha^c| \geq 1\}\,.$$

In general, we can say that $u \equiv_{\pi_\alpha} v$ if u and v have the same colour.

2.5.1 Partition refinements

We now define refinement of partitions, which is a generalisation of the way that colour refinement subdivides the colour classes of a graph.

Definition 2.22 (Partition refinement)**.** For two partitions π and ρ of V, we say that ρ *refines* π if for all $u, v \in V$:

$$u \equiv_\rho v \Rightarrow u \equiv_\pi v\,.$$

If, in addition, π is not equal to ρ, then ρ *strictly refines* π.

Intuitively, a partition ρ refines a partition π if it subdivides the equivalence classes of π. We illustrate partition refinement with two partitions in the following example.

Example 2.23. *Figure 2.9 depicts the graph G, with its degree colouring α_{degree}. Consider the following two partitions of the vertices of G*

- *The partition of nodes into colour classes $\pi_{\alpha_{degree}}$ in Figure 2.10a. As described in Definition 2.21, we can define a partition of nodes into colour classes. Since there are two different colours in α_{degree}, this results in a partition of two cells.*
- *The partition ρ in Figure 2.10b. ρ is a partition of three cells.*

The partition ρ refines $\pi_{\alpha_{degree}}$, since for any combination of two vertices u and v, u and v are only in the same red cell in Figure 2.10a if they are in the same blue cell in Figure 2.10b. Intuitively, we can describe this by saying that all blue cells are subcells of a red cell.

In this case, ρ also strictly refines $\pi_{\alpha_{degree}}$, since $\pi_{\alpha_{degree}} \neq \rho$.

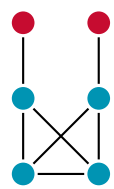


Figure 2.9: Graph G from Example 2.23 with its degree colouring.

(a) Partition $\pi_{\alpha_{\text{degree}}}$ of G.

(b) Partition ρ of G.

Figure 2.10: Example partitions of $V(G)$ in Figure 2.9

The colour refinement algorithm in each iteration creates a partition (of vertices into colour classes) that refines the partition from the previous iteration. It is exactly this notion of partitions that are the outcome of colour refinement that is unique. We make this more formal in Section 2.5.2.

2.5.2 Coarsest Stable Partition

We first translate a partition to a colouring (the opposite direction of Definition 2.21).

Definition 2.24. Let $\mathcal{G}$ be a set of graphs and π be a partition of $V(\mathcal{G})$. We define α_π, the *colouring of the cells*, to be the colouring such that for every $u, v \in V(\mathcal{G})$, we have

$$\alpha_\pi(u) = \alpha_\pi(v) \quad \iff \quad u \equiv_\pi v\,.$$

Since this definition translate any partition to a colouring, it is intuitive to also define a *stable partition*.

Definition 2.25. Let $\mathcal{G}$ be a set of graphs and π be a partition of $V(\mathcal{G})$. We say that π is *stable* if the colouring of the cells is a stable colouring.

There can exist multiple stable partitions for a certain set of vertices $V(G)$. We now define a qualification that a stable partition can have:

Definition 2.26. Let π be a stable partition of $V(G)$. π is called a *coarsest stable partition* of $V(G)$ if there is no other stable partition ρ of $V(G)$ such that π strictly refines ρ.

Analogously, when an initial partition π^0 is given, we define the *coarsest stable partition that refines* π^0.

We now consider the question if a coarsest stable partition is unique or not.

Theorem 2.27. *For every partition π^0 of $V(\mathcal{G})$, there is a* unique *coarsest stable partition of $V(\mathcal{G})$ that refines π^0, which is exactly the partition that is the result of the colour refinement algorithm, when starting with π^0.*

Proof. For this proof, we ignore the colours assigned by colour refinement, and just consider the corresponding partitions as defined by Definition 2.21 for each of the colourings, $\pi^0, \ldots, \pi^l$. Assume that ρ is a coarsest stable partition that refines π^0. We proof by mathematical induction that ρ refines each of the partitions $\pi^0, \ldots, \pi^l$. We then conclude that $\rho = \pi^l$ since both are stable, ρ refines π^l and all other stable partitions refine π^l as well.

Base step: ρ refines the start partition π^0.

Induction hypothesis (IH): ρ refines the partition π^i.

Induction step: By (IH), ρ refines π^i, so if $u \equiv_\rho v$, then $u \equiv_{\pi_i} v$. Now if ρ does not refine π^{i+1}, we have that there exist $u, v \in V(\mathcal{G})$ s.t. $u \equiv_\rho v$ and $u \not\equiv_{\pi_{i+1}} v$. However, this means that ρ is not stable since π_{i+1} distinguishes u and v and, therefore, they have differently-coloured neighbourhoods in both π^i and ρ. Thus, ρ must refine π^{i+1} which completes the proof. □

We conclude that, indeed, the coarsest stable partition is unique and it is the unique outcome of the colour refinement algorithm.

2.6 Relation to the #Aut problem

Chapter 1 introduced the #Aut problem. Recall that the #Aut problem asks, for a given graph G, how many automorphisms G has (isomorphisms from G to itself).

First, note that *any* graph G with $|V(G)| = n$ vertices has at least one automorphism: $f(v) = v, \forall\, v \in V(G)$, the trivial automorphism that maps each vertex to itself. On the other hand, it has at most $n!$ automorphisms, since each automorphism is a permutation and there are $n!$ permutations of n vertices. For example, the complete graph K_n has exactly $n!$ automorphisms (as does the edgeless graph on n vertices). Chapter 3 and 5 discuss automorphisms in more detail, but we can already conclude a few things from what we know from this chapter.

In Section 2.4, we have drawn conclusions about the existence of an isomorphism from G to H. Additionally, we concluded that in those cases where we can conclude that an isomorphism exists it is unique (see Observations 2.18 and 2.19).

Since an automorphism is an isomorphism from G to G itself, we can apply the colour refinement algorithm on G and adjust the observations to the new context. This lets us conclude the following with respect to automorphisms:

Observation 2.28. *Let α be the coarsest stable colouring of G that refines $\pi_{\alpha_{\text{uniform}}}$, the partition of nodes into colour classes of the uniform colouring. Then:*

a) Any automorphism of G is colour preserving, with respect to α.

b) If α is discrete, then G has only one automorphism.

Note that by Observation 2.28b), we have solved the #Aut problem for the cases where α is discrete. When α is not discrete, we cannot draw a conclusion regarding the number of automorphisms of G. Chapter 3 discusses a first approach to solve the #Aut problem.

Chapter 3

Branching

3.1 Intuition

Chapter 2 introduced the concept of colour refinement and how we can use it to determine whether two graphs are isomorphic to each other. If the coarsest stable colouring of two graphs is both balanced and discrete, we that a bijection between the two graphs exists, and that the two graphs are isomorphic. See, for example, the two graphs from Figure 3.1. The colouring of these two graphs is balanced and a bijective mapping from every vertex in the first graph to the vertices of the second graph exists, which is therefore an isomorphism.

If we now look at the two graphs in Figure 3.2. We see that even though they have a balanced and stable colouring, the colouring is not discrete. We cannot say it is a bijection, however there might exist one, at this point we can simply not conclude whether the two graphs are isomorphic. In this chapter we look at a so-called "branching method" to see how we can efficiently determine if there exists an isomorphism, when two graphs have a stable, balanced but not discrete colouring.

In the case of Figure 3.2 the two graphs are isomorphic however that is not always necessarily true, consider the two graphs from the Figure 3.3.

Consider the two graphs in Figure 3.3. We coloured the vertices of these graphs based on their degree, you can check yourself that this colouring is stable. when looking closely at these graphs, we see that these graphs are not isomorphic to each other. So as mentioned before, we need to look at a method to determine whether two graphs are isomorphic if we know that the two graphs have a stable and balanced colouring that is not discrete.

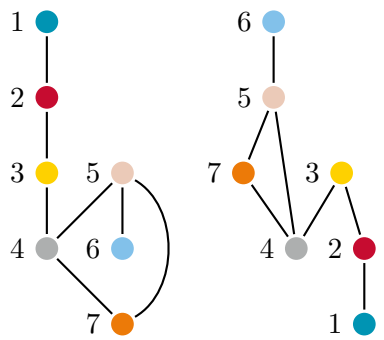


Figure 3.1: Two isomorphic graphs with a stable balanced discrete colouring. Labels are colour numbers

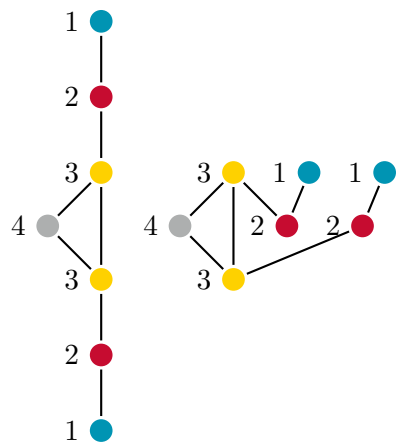


Figure 3.2: Two isomorphic graphs with a stable balanced non-discrete colouring.

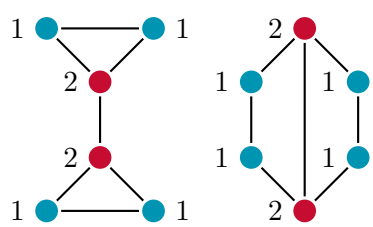


Figure 3.3: Two non-isomorphic graphs with a stable balanced colouring.

3.1.1 The method

Consider a colouring α, where α is the stable colouring of G and H that results from applying colour refinement on both graphs where all colours "have the same meaning", starting with the *uniform* colouring. For some graphs we encounter a problem if we wish to determine whether the two graphs are isomorphic; the coarsest stable colouring α is balanced however it does not define a bijection. We could simply try every possible colour preserving bijection. However, in general this is not a good idea as for different graphs this might lead to unreasonable many possibilities which is undesirable. Another way we could approach this is to choose a vertex x in graph G whose colour is not unique in G. We then map this vertex in G to a vertex y in graph H with the same colour as vertex x. We colour vertex x and y a new colour which has not been used in either graph and again apply colour refinement on the graph, we call this “branching”.

Let us look at the the graphs in Figure 3.4. We first map vertex x in graph G to vertex y_1 in graph H. We colour both vertices a new unused colour and then colour refine the two graphs again. If we have a bijection of G onto H after “branching” on vertex x and y_1 we can determine that the two graphs are isomorphic. If the colouring is unbalanced after branching, we then have to repeat this but we go back to the graphs in Figure 3.4 and instead we map vertex x in graph G to vertex y_2 in graph H. This in fact also gives a isomorphism between G and H. Interestingly there exist two isomorphisms between G and H, we discuss more about this in Section 3.3 when solving the *#Aut problem*.

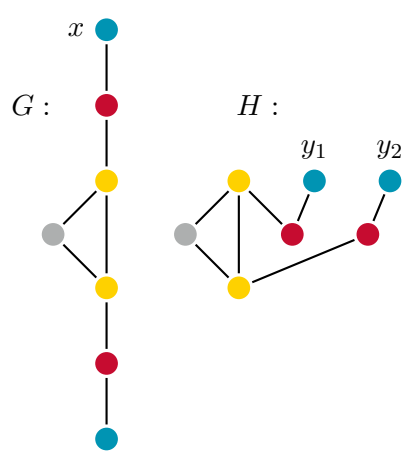


Figure 3.4: Two graphs with a stable and balanced colouring.

We will now look at a more systematic and concrete description of branching.

1. Suppose α is a balanced stable colouring of $\{G, H\}$ but α does not define a bijection.

2. There exists a colour class C with $|C| \geq 4$. Denote by $C_G = C \cap V(G)$ the set of vertices in G that have colour C and by $C_H = C \cap V(H)$ the set of vertices in H that have colour C. Since α is balanced, $|C_G| = |C_H| = k$, with $k \geq 2$.

3. Choose an arbitrary $x \in C_G$. Denote $C_H = \{y_1, \ldots, y_k\}$. Every colour preserving isomorphism f has $f(x) = y_i$ for some $i \in \{1, \ldots, k\}$.

4. Try all possibilities, $f(x) = y_i$, for $i \in \{1, \ldots, k\}$. This gives k branches of a recursive algorithm. There exists a colour preserving isomorphism if and only if a colour preserving isomorphism is found in at least one branch.

5. In each branch, the choice $f(x) = y_i$ is encoded by giving both vertices a *new, unique* colour.

6. Given this initial colouring, we again apply colour refinement and continue recursively until either an isomorphism is found, or we conclude that no isomorphism with $f(x) = y_i$ exists.

3.2 Example

To get a better understanding of how *branching* works, we will now look at an example of how branching works on two graphs G and H. Consider the following graph in Figure 3.5 with a uniform stable colouring:

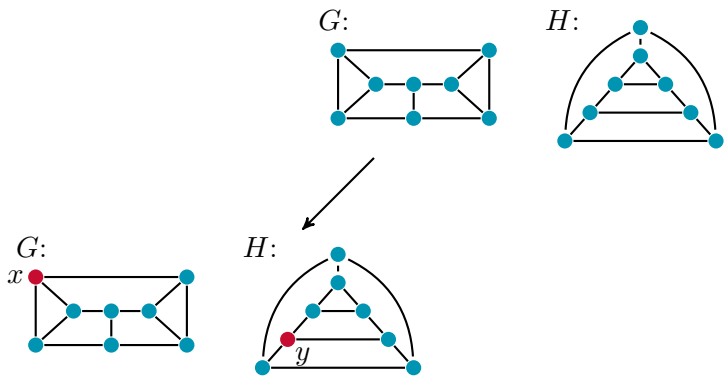


Figure 3.5: Two graphs with a uniform colouring, where we map a vertex x from G to vertex y from H and both are given a new distinct colour, in this case red.

From graph G we chose a vertex x with a colour where there are multiple vertices in G with the same colour as x, from graph H we chose a vertex y which has the same colour as x. We then make these two vertices the same, but a new and unique colour, red instead of blue in this example. We then again refine these graphs given the new colouring we got from branching on x and y, as shown in Figure 3.6:

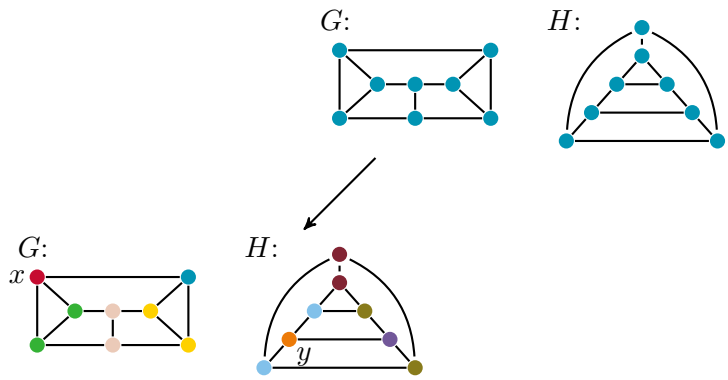


Figure 3.6: Two graphs with a uniform colouring, where we map a vertex x from G to vertex y from H and both are given a new distinct colour red. We executed colour refinement on the graphs to show that there does not exist an isomorphism from G to H where x is mapped to y.

From Figure 3.6 we can easily see that this is not a balanced colouring as the colours in both graphs are completely different. We know that if there exists a bijection from G to H that vertex x can not be mapped to y, so we have to see whether x can be mapped to a different vertex in H. As shown in the figure below: We return to the original colouring of the graphs, where all vertices are blue, and we keep vertex x to be the same vertex, however we now choose a different vertex in H and again call it y, where y has the same colour as x. Both vertices are made a new unique colour, again we make them red instead of blue.

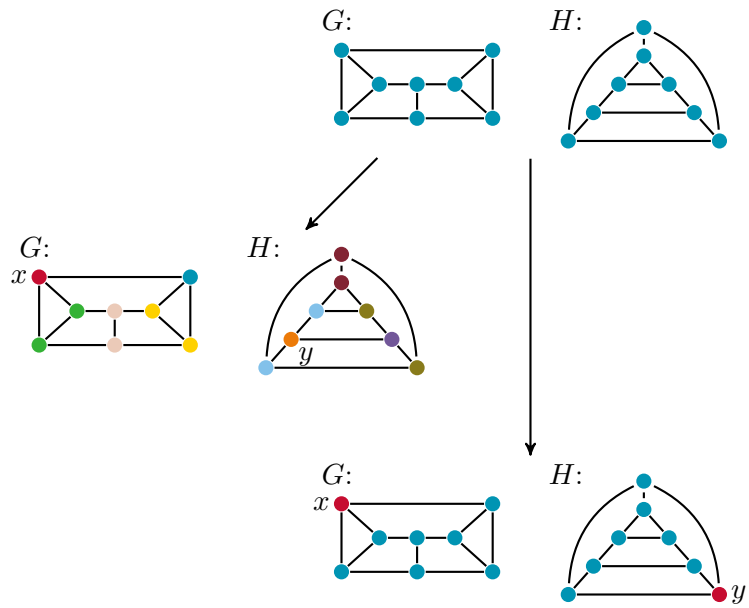


Figure 3.7: Two graphs with a uniform colouring, where we map the same vertex x from G to a different vertex y from H and both are given a new distinct colour red.

From Figure 3.8, we again see that after partitioning on the new colouring the colouring becomes unbalanced, so vertex x can also not be mapped to this vertex y. We keep branching on all vertices from graph H with the same colour as x until we either find a bijection or we conclude that the two graphs are not isomorphic. Now consider the following choice of vertex y as shown in Figure 3.8.

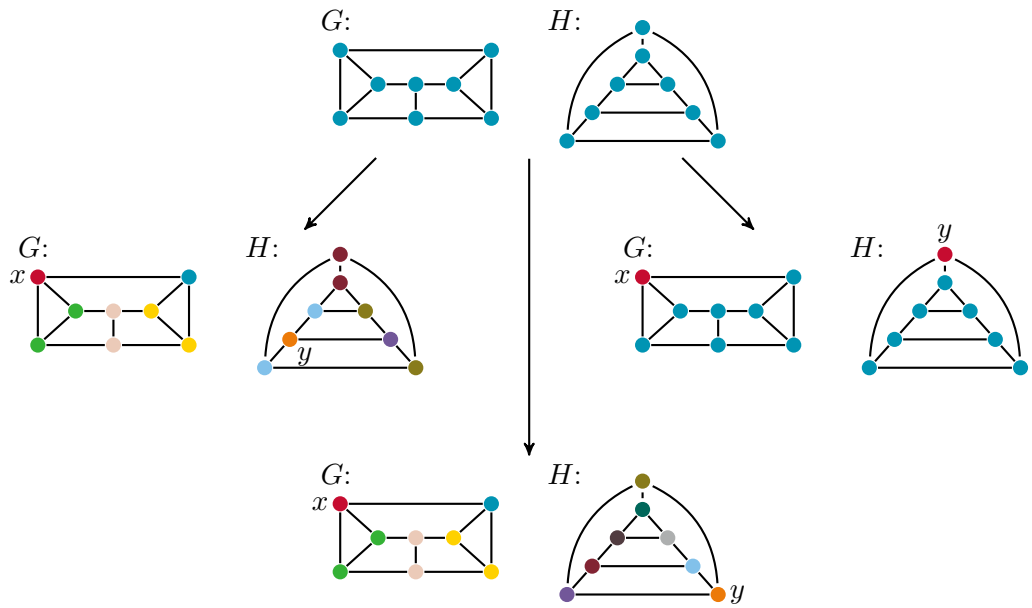


Figure 3.8: Two graphs with a uniform colouring, where we map the same vertex x from G to a different vertex y from H and both are given a new distinct colour red. We executed colour refinement on the graphs to show that there does not exist a bijection from G to H, where x is mapped to this new y. We also map the same vertex x from G to an again different vertex y from H and both are given a new distinct colour red.

In Figure 3.9 we see that after branching on this specific choice of vertices x and y, that the new stable colouring is balanced but not yet discrete. So, same as what we did at the very beginning. We choose a vertex x_2 from graph G who's colour is **not** unique in G and we chose a vertex from H with the same colour as x_2 and call it y_2. We also give x_2 and y_2 unique colours, in this case we make them purple, and again partition on this new colouring.

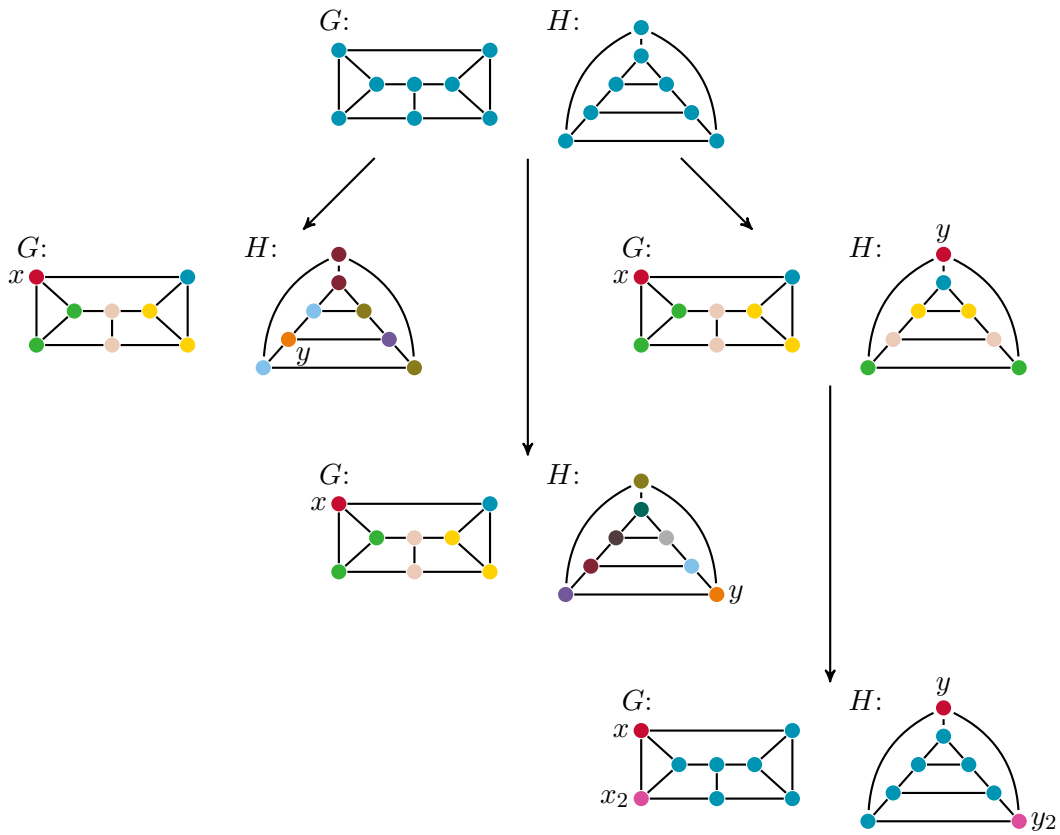


Figure 3.9: Two graphs with a uniform colouring, where we map the same vertex x from G to a different vertex y from H and both are given a new distinct colour red. We executed colour refinement on the graphs to show that there does not exist a bijection from G to H, where x is mapped to this new y. We also map a new vertex x_2 from G to an vertex y_2 from H in the same green colour class, both are given a new distinct colour purple.

And finally in Figure 3.10 we can see that in this branch there indeed does exit a bijection from G onto H, hence G and H are isomorphic.

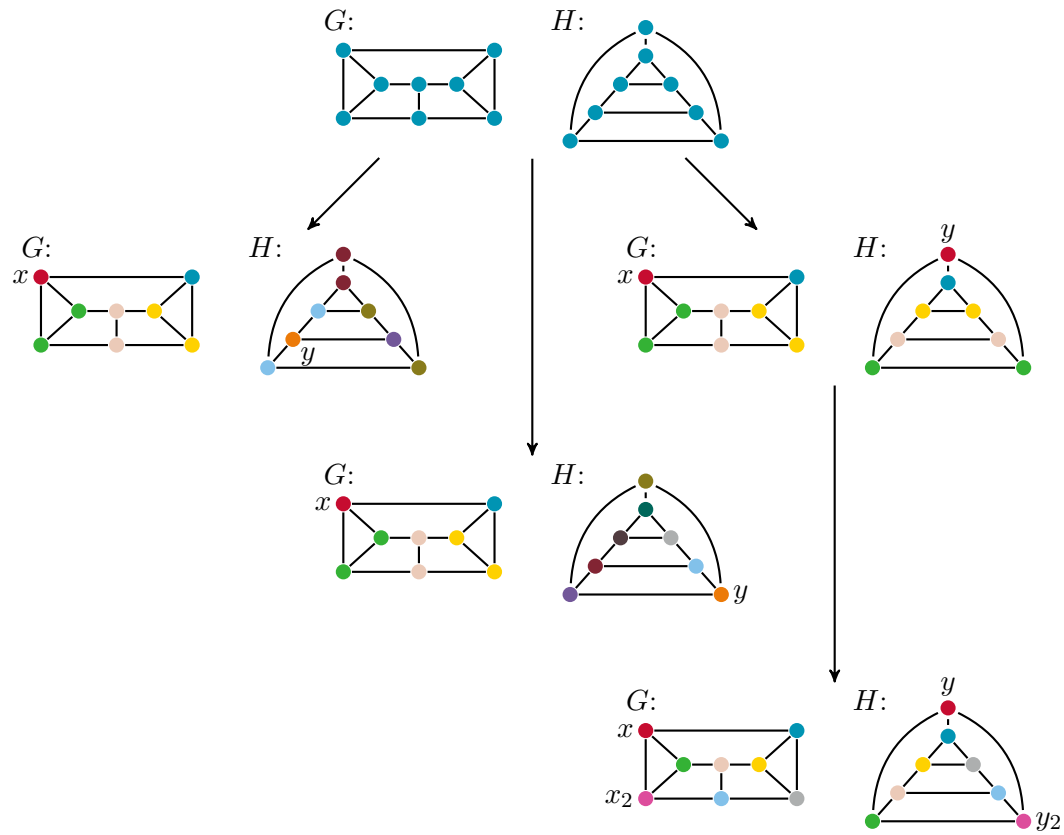


Figure 3.10: Two graphs with a uniform colouring, where we map the same vertex x from G to a different vertex y from H and both are given a new distinct colour red. We executed colour refinement on the graphs to show that there does not exist a bijection from G to H, where x is mapped to this new y. We also map a new vertex x_2 from G to an vertex y_2 from H in the same green colour class, both are given a new distinct colour purple. Who, when refined, give a bijection.

3.3 #Aut Problem

Something we are also interested in, is the "counting automorphisms problem" or the "*#Aut problem*". For this we will re-introduce the concept of an automorphism. Remember the definitions given in Section 1.2.

Definition 3.1. An automorphism of a graph $G = (V, E)$ is a permutation σ of the vertex set V, such that the pair of vertices (u, v) form an edge if and only if the pair $(\sigma(u), \sigma(v))$ also form an edge. That is, it is a graph isomorphism from

G to itself.

Observation 3.2. *Every graph G has at least one automorphism, namely the trivial permutation.*

This is quite obvious as we can very simply define a bijection f which maps G to G as $f(v) = v$ where v is a vertex of G. We are interested in finding $Aut(G)$ which is the set of all automorphisms of G, and more specifically we want to know $|Aut(G)|$.

Theorem 3.3. *If G is isomorphic to H, then $Aut(G) = Aut(H)$ and $|Aut(G)| = |Aut(H)|$.*

Proof. Suppose G is isomorphic to H, this means there exists a bijection f which maps $V(G)$ to $V(H)$. We know that if the graph Q_1 is isomorphic to Q_2 and Q_2 is isomorphic to Q_3 then Q_1 must be isomorphic to Q_3. We can use this to say that every automorphism of G is therefore also isomorphic to H and since $Aut(H)$ is the set of all isomorphisms of H onto H we know that $Aut(G) \subseteq Aut(H)$. However, this is also true the other way around, that is, $Aut(H) \subseteq Aut(G)$. This means that $Aut(G) = Aut(H)$ and therefore also $|Aut(G)| = |Aut(H)|$. □

The method of solving this problem is not very different to the isomorphism problem. For the *#Aut problem* we instead of having two different graphs G and H, we take H to be the same graph as G. This means that we are basically comparing two identical graphs so of course we will determine that they are isomorphic to each other. However we keep checking all branches for possible isomorphisms, instead of stopping after finding a single one, while keeping track of how many isomorphisms we found, we find $|Aut(G)|$.

Consider the example from Section 3.2. Here we stopped our branching algorithm after finding a single isomorphism. If we now also map the green vertex x_1 from G to both green vertices of H we know that at least one of them is an isomorphism but we might find that both mappings of x_1 to the green vertices of H are isomorphisms. However there might also be more isomorphisms for mapping x to different blue vertices in H. In the next section we will discuss an algorithm that can solve this problem and prove that this algorithm also returns the correct amount of automorphisms.

3.4 Implementation Details

Now that we have seen an example of the branching algorithm, we look at how to implement it using the colouring refinement algorithm. Let $D = (x_1, \ldots, x_d)$

and $I = (y_1, \ldots, y_d)$ be sequences of distinct vertices of G and H respectively. We know that a graph G and H are isomorphic to one another if there exists a bijection $f : V(G) \to V(H)$.

Definition 3.4 (Bijection following (D, I))**.** Given two order sequences $D = (x_1, \ldots, x_d)$ and $I = (y_1, \ldots, y_d)$, we say that a bijection $f : V(G) \to V(H)$ *follows* (D, I) if $f(x_i) = y_i$, for all $i \in \{1, \ldots, d\}$.

Intuitively, a bijection from the vertices of G to the vertices of H that satisfies Definition 3.4 is restricted to map exactly the first element from D to the first element of I, and the same for all the other $d - 1$ elements of D. This definition allows us to talk about bijections which we partly define beforehand. Let us say that, for whatever reason, we know for certain that $v \in V(G)$ has to be mapped to $u \in V(H)$ in any isomorphism from G to H. Then, we can define $D = (v)$ and $I = (u)$ and only consider bijections that follow (D, I).

We now define a colouring in such a way that, if we refine that colouring, we ensure that any bijection that is defined by the coarsest stable colouring that refines it follows (D, I). Let $\alpha(D, I)$ be the colouring of $\mathcal{G} = \{G, H\}$ that assigns colour i to x_i and y_i for $i \in \{1, \ldots, d\}$, and colour 0 all other vertices of G and H.

Proposition 3.5. *Let β be the coarsest stable colouring of $\mathcal{G} = \{G, H\}$ that refines α(D,I). Then every isomorphism f from G to H that follows D,I is β-colour preserving.*

Corollary 3.6. *If β is unbalanced, no isomorphism from G to H exists that follows (D, I). If β defines a bijection f, then f is the unique isomorphism from G to H that follows (D, I).*

Consider the following pseudo-code for a subroutine that counts the number of isomorphisms from G to H that follow (D, I).

Note that in this algorithm $D + x$ implies adding vertex x to the set of vertices D.

Theorem 3.7. *Algorithm 2 returns the correct number of isomorphisms from G to H that follow (D, I).*

Proof. Consider a recursion tree resembling the all pairs of (x, y) that Algorithm 2 visits. Then for the leaves of this recursion tree:

- If β is unbalanced, the algorithm correctly returns 0.
- If β defines a bijection, the algorithm correctly returns 1.

Algorithm 2 CountIsomorphism

Input: Sequences D and I, which are equal length sequences of vertices of G and H respectively.
Output: The number of isomorphisms from G and H that follow (D, I).

Compute the coarsest stable colouring β of $\mathcal{G} = \{G, H\}$ that refines α(D,I).

if β is unbalanced **then**
  return 0
if β defines a bijection **then**
  return 1

Choose a colour class C with $|C| \geq 4$.
Choose $x \in C \cap V(G)$.
num = 0
for $y \in C \cap V(H)$ **do**
  num := num + **CountIsomorphism**$(D + x,\ I + y)$
return num

Induction hypothesis (IH): Assume the algorithm correctly counts isomorphisms for all pairs of vertices (x, y) at a given depth in the recursion tree.

Induction step: Consider the recursive step where the algorithm chooses a colour class C with $|C| \geq 4$ and a vertex x from G in that colour class. Then Algorithm 2 iterates over vertices y from H in the same colour class, calling *CountIsomorphism* recursively for each pair (x, y) with extended sequences $D+x$ and $I+y$. According to the inductive hypothesis, the recursive calls for each pair (x, y) correctly count the number of isomorphisms. The algorithm explores all possible pairs of vertices in G and H for the chosen colour class.

Combining the base case and the inductive step, we can conclude that Algorithm 2 correctly counts the number of isomorphisms from graph G to graph H that follow the sequences D and I.

Therefore, by induction over the recursion tree, Algorithm 2 is proven to be correct in calculating the correct amount of isomorphisms between G and H. □

Theorem 3.8. *Algorithm 2 terminates.*

Proof. Consider 2 graphs G and H, let $V(G) = \{x_1, x_2, \ldots, x_n\}$ denote a sequence of all vertices from graph G and $V(H) = \{y_1, y_2, \ldots, y_n\}$ denote a sequence of all vertices from graph H. Let $D = (x_1, \ldots, x_d)$ and $I = (y_1, \ldots, y_d)$ be sequences of distinct vertices of G and H and $\alpha(D, I)$ be the colouring of $\mathcal{G} = \{G, H\}$ that assigns the colour i to x_i and y_i for $i \in \{1, \ldots, d\}$, and colour 0 all other vertices of

G and H. When $|D| = |I| = |V(G)| = |V(H)| = n$ the algorithm terminates, with every iteration of the algorithm $|D|$ and $|I|$ increase by one, so given any D and I the recursion depth is at most $|V(G)| = n$ and thus the algorithm terminates. □

One should note that if we only need to determine whether two graphs are isomorphic we can stop the algorithm the moment we find a single bijection. This might be obvious but is often forgotten, if we want to determine whether two graphs are isomorphic we should not calculate all possible isomorphism as a single one is sufficient and this will drastically increase the computation time of the algorithm

Now to come back to the *#Aut problem*, if we want to determine the amount of isomorphisms a graph G has to itself we can still use Algorithm 2, but instead if having two distinct graphs as our input we take $H = G$ and start comparing graph G to itself. We have already proven that this algorithm returns the correct amount of isomorphisms from G to H, this then also implies that if we take $H = G$ to be our input then this algorithm returns the correct amount of isomorphism from G to G, also know as the number of *automorphism*.

3.5 Improvements

It is possible to improve the computation time of the algorithm by choosing the colour class C on which you branch cleverly. This particular choice is called a *branching rule*. An example of a branching rule is by preferring colour classes that contain vertices with a high or small degree, or colour classes that contain many or few vertices, or make a decision based on the order in which colour classes were discovered, or something else.

It is difficult to decide which branching rule we need to choose for the best computation time, as the results might be different for different types of graphs. It is up to you to try out several different branching rules and find one or more rules that performs better than straightforward rules and support this with computational experiments.

In previous examples in this chapter we went back to an uniform colouring every time we branched on a vertex. Instead of going back to the uniform colouring we could also keep the colouring of the graphs at the moment of branching. This reduces the amount of iterations of the colour refinement we have to execute to achieve a stable colouring. When we do this we then do need to store the colouring of the graph before you branch, so that when you are done branching on a vertex you can return to the colouring you branched from.

Chapter 4

Fast Colour Refinement

4.1 Intuition

Chapter 2 treats an algorithm that finds the coarsest stable colouring of a graph with a complexity of $O(n^2 m)$, where n is the number of vertices and m the number of edges in the graph. While this scales quite well, it can be improved. This chapter introduces an algorithm with runtime complexity $O(m \log n)$. It turns out to be especially fast when a graph contains long paths.

We explain this algorithm through a problem [1] that is historically related and that you know from your Languages and Machines (L&M) course, DFA minimisation. The first section explains the basic algorithm to minimise DFAs using partition refinement. The second section explains the improved DFA minimisation algorithm with complexity $O(m \log n)$. The final section translates the algorithm for use with the GI-problem and highlights some things to consider for implementation.

4.2 Minimising Deterministic Finite Automaton

As in your L&M course, a deterministic finite automaton D is represented by a quintuple, $(Q, \sum, \delta, q_0, F)$, consisting of

- a finite set of states Q,
- an alphabet $\sum$,
- a transition function $\delta : Q \times \sum \to Q$,
- an initial or starting state $q_0 \in Q$, and
- a set of accept states $F \subseteq Q$.

Definition 4.1 (Equivalent States)**.** Two states q_i and q_j are equivalent if they

accept the same language. That is, $\mathcal{L}\big((Q, \sum, \delta, q_i, F)\big) = \mathcal{L}\big((Q, \sum, \delta, q_j, F)\big)$, or

$$\hat{\delta}(q_i, w) \in F \iff \hat{\delta}(q_j, w) \in F\,.$$

In other words, two states (q_i, q_j) are **not** equivalent if and only if there exists a word w that distinguishes them. Which means that only one results in an accepting state.

4.2.1 Outline of the Minimisation

The goal is to find the smallest (least number of states) DFA that accepts the same language as the given DFA. We do this by combining equivalent states into a single state. Thus, we want to group all equivalent states. Dividing all states into groups is a partition. Therefore our goal can be rephrased as follows.

Goal	Partitioning all states by equivalence.
Method	*1.* Start with a valid partition.
	2. Refine this until it is stable.

This method solves DFA minimisation, but we do not have to limit ourselves to only one DFA. We can also apply this same method to two DFAs simultaneously. Which also allows us to do DFA equivalence testing.

Observation 4.2 (DFA Equivalence)**.** *Applying the method above to two DFAs D_1 and D_2. Then $\mathcal{L}(D_1) = \mathcal{L}(D_2)$ if and only if their two start states are equivalent.*

This means that, as long as both your starting states end up in the same partition, both DFAs accept the same language. Which is, by definition, that the two DFAs are equivalent.

4.2.2 Partition Notation

To work with a partition or colouring of the states Q of a DFA, we need some notation.

4.2.3 The Method of Minimisation

The Invariant

Before we introduce the method, we define an invariant. This is a statement that remains true for each partition that our Algorithm (intermediately) produces. The invariant ensures that the program terminates with the right outcome.

Symbol	Name	Description	Math Notation
i	colour	an index that keeps track of the colour of a state	$i \in \{0, \ldots, k-1\}$
C_i	colour class	a set of states that currently have the same colour i	$C_i = \{q_n, \ldots, q_m\}$
π	current partition	a set of colour classes that partitions all states	$\pi = \{C_0, \ldots, C_{k-1}\}$
$\equiv_\pi$	colour equivalence	denotes that states are part of the same colour class in π	$q_i \equiv_\pi q_j$

Table 4.1: Partition or Colouring Notation

Invariant 4.3. *If $q_i \not\equiv_\pi q_j$, there exists a word w that distinguishes q_i from q_j.*

In other words, when two states are not in the same colour class, there is a word that distinguishes them. Note that the following disposition is **not** true: if two states are in the same colour class, then they can't be distinguished. When the former also holds, we have a stable partition and we are done. However, if we keep refining our partition, then we will eventually end up with the partition where it is the case.

To start our operation with an initial partition. We could start with the partition that contains all states in one colour class (similar to the uniform colouring), but for convenience of the presentation we start with another simple partition that always satisfies the invariant.

Observation 4.4. *The partition $\{F, Q \setminus F\}$ satisfies the invariant.*

Proof. The empty word λ distinguishes accepting from nonaccepting states. □

The Refining Operation

The intuition behind our operation is based on our invariant. When the last symbol of a word results in a different state, then those states can be distinguished. Thus they can be put into different colour classes. Let us say we take all states within a colour class and let them transition with the same symbol. If they then do not all end up in a single (may be different) colour class, they can be distinguished. Therefore we can split up our original picked colour class while maintaining the invariant.

Lemma 4.5 (Operation Refine(C, x)). *Let C be a colour class in the current partition. Let $x \in \sum$ be a symbol. Let $A \subseteq Q$ be all states q_i with $\delta(q_i, x) \in C$.*

Algorithm 3 REFINE(C, x)

Init: Given graph G and current partition π_i

Let $A = \{\, q \mid \delta(q, x) \in C \,\}$
Set $\pi_{i+1} = \{\}$
for D a colour class in π_i **do**
  $D_A, D_{\overline{A}} = D \cap A, D \setminus A$
  $\pi_{i+1} = \pi_{i+1} + \{D_A, D_{\overline{A}}\}$

Then replacing each cell D in the current partition by $D_A = D \cap A$ and $D_{\overline{A}} = D \setminus A$, provided both are nonempty, maintains Invariant 4.3.

Proof. Consider $q_i \in D_A$ and $q_j \in D_{\overline{A}}$. Let $q_i' = \delta(q_i, x)$ and $q_j' = \delta(q_j, x)$. So $q_i' \in C$, but $q_j' \notin C$. According to the invariant, there exists a word w such that exactly one of $\hat{d}(q_i', w)$ and $\hat{d}(q_j', w)$ is accepting. Then also exactly one of $\hat{d}(q_i', xw)$ and $\hat{d}(q_i', xw)$ is accepting. In other words, the word xw distinguishes q_i and q_j. Doing this for all state pairs shows that the invariant is maintained. □

From this point on, the operation REFINE(C, x) is called the *refining operation using C and x*. By REFINE(C) we denote the operation that applies REFINE(C, x) for all $x \in \sum$.

Algorithm 4 REFINE(C)

Init: Given graph G and current partition π_i

for $x \in \Sigma$ **do**
  REFINE(C, x)

Note that the operation REFINE(C, x) has no effect if, for all colour classes D, either $D \cap A = \emptyset$ or $D \setminus A = \emptyset$.

DEFINITION 4.6 (Stable Partition)**.** Let C be a colour class of the current partition π. We say that π is stable for C if the operation REFINE(C) has no effect. $\pi = \{C_0, \ldots, C_{k-1}\}$ is stable if π is stable for all C_i.

Note that we can define REFINE(C) on a partition π even if C is not a colour class of π. Similarly, we can say that π is stable for C even if C is not a colour class of π.

THEOREM 4.7. *Algorithm 5 terminates with the (unique) partition into equivalent states.*

Algorithm 5 The Basic Algorithm of Minimising DFAs

Init: Create the partition $\pi_0 = \{F, Q \setminus F\} = \{C_0, C_1\}$

$i = 0$.
while π_i is not stable **do**
  Pick a C_j and x such that π_i is refined by $\textsc{Refine}(C_j, x)$
  Apply $\textsc{Refine}(C_j, x)$ to π_i and let its output be π_{i+1}
  $i = i + 1$

In π_i, identify all states in the same colour class

Proof. First note that, if π_i is not stable, there are C and x such that applying $\textsc{Refine}(C, x)$ increases the number of colour classes in the partition (i.e., $|\pi_{i+1}| > |\pi_i|$). Since $|\pi| \leq |Q|$ for all partitions π, Algorithm 5 eventually terminates.

Let π^A be the partition produced by Algorithm 5, and π^L be the partition into equivalent states. We show that $\pi^A = \pi^L$, by proving two directions.

π^L refines π^A: The initial partition $\{F, Q \setminus F\}$ satisfies the invariant. By Lemma 2.11, the final partition π^A satisfies the invariant, and is therefore refined by π^L.

π^A refines π^L: Suppose to the contrary that there exists a word w that distinguishes two states q_i and q_j with $q_i \equiv_{\pi^A} q_j$. Choose w, q_i and q_j such that w is a *shortest word* with these properties. Clearly, $|w| \geq 1$ since we start with the partition $\{F, Q \setminus F\}$. So let $w = xv$ for $x \in \Sigma$, and let $q_i' = \delta(q_i, x)$ and $q_j' = \delta(q_j, x)$. Since π^A is stable, there exists a colour class C' such that $q_i', q_j' \in C'$. However, the word v distinguishes q_i' from q_j', and v is shorter than w, which contradicts that w is the shortest word. □

With the outcome of Algorithm 5, we merge all states in one equivalence class into a single state of our new DFA. We do this for each equivalence class. The transitions between the equivalence classes are easily (and uniquely) deduced. If a transition is not easily deduced, e.g., because two state in the same colour class transition into different colour classes with the same symbol, the partition was not stable.

4.2.4 An Example

While Algorithm 5 may seem complicated, when applying it in practice, one quickly gets an intuition for it. Consider the example in Figure 4.1. Figure 4.3 shows the steps of Algorithm 5. To make the process clearer, we keep track of the colour classes. Furthermore, we specify for each $\textsc{Refine}()$ operation what colour class(es) split up.

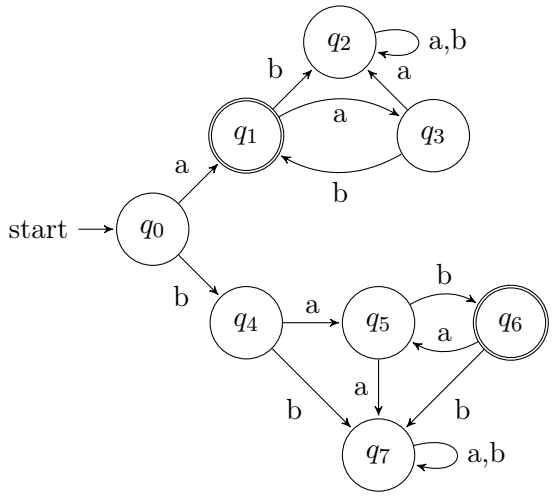


Figure 4.1: Example DFA Colouring

4.3 Faster Minimisation Algorithm

Only implementing the Refine() operation from Algorithm 5 does not achieve the goal that we set for ourselves all by itself. For that we need be careful how we exactly implement it.

4.3.1 Stable and Unstable Colour Classes

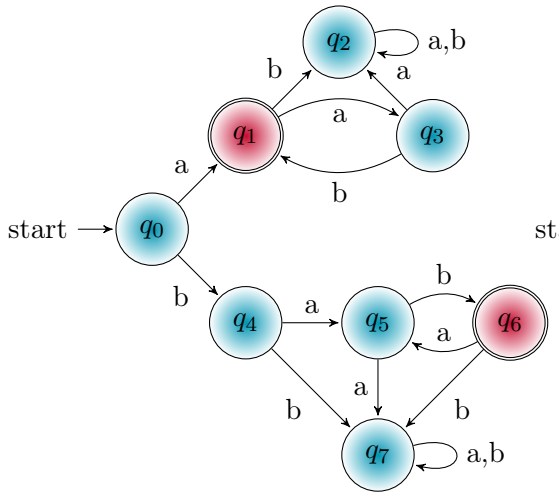
When we refine with Colour C, our partition π may be stable for the Colour C' that we refine or not. Note that, if π is stable for C, refining with C has no effect on π.

Observation 4.8. *If π' refines π, and π is stable for C, then π' is stable for C.*

Note again that π' can be stable for C, even if C is not a colour class in π' (anymore). See for example Figure 4.4b.

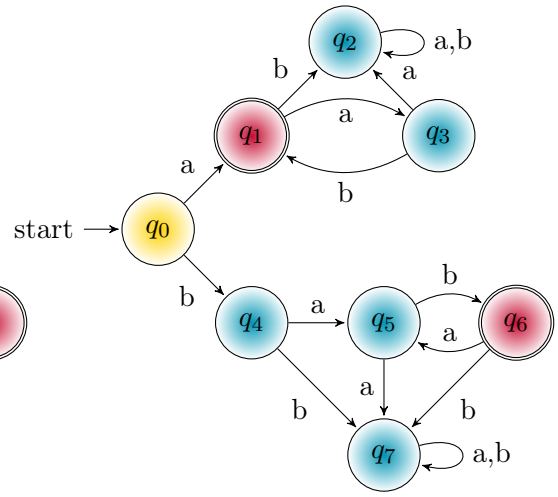
If π is stable for C, this means that for all $x \in \sum$, the states in all other colour classes either all transition into C with x or none do so. So we are stating: Given any x, if in all other colour classes, either all the states transition into C with x or none of the states do. Then, refining our partition does not change this fact. Figure 4.4 shows an example of this observation.

With this observation, we can prove the following Lemma 4.9, which will allow us to improve our computation time significantly.



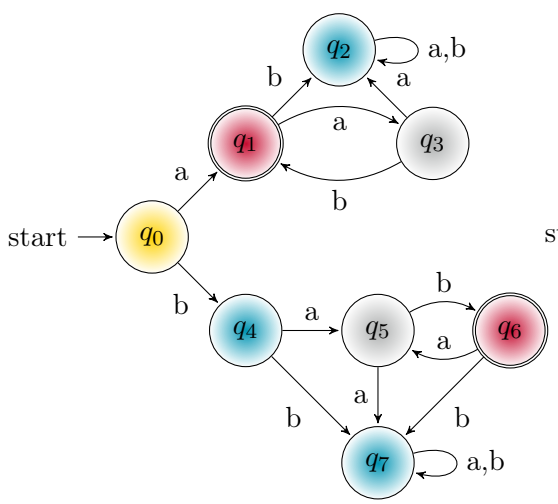
Colour:	states:
0	q_1, q_6
1	$q_0, q_2, q_3, q_4, q_5, q_7$

(a) Refine(C_0, a): $A = \{q_0\}$. 1 splits.



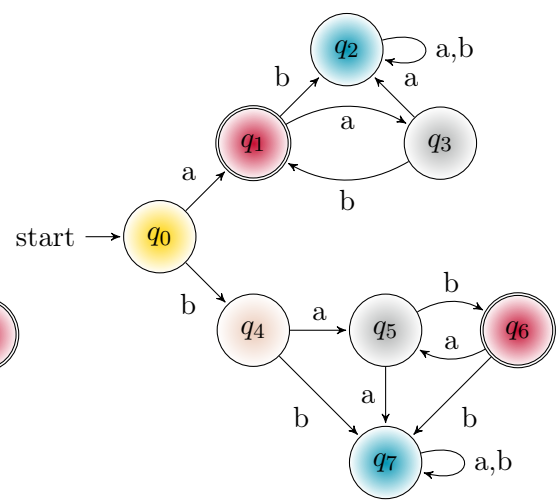
Colour:	states:
0	q_1, q_6
1	q_2, q_3, q_4, q_5, q_7
2	q_0

(b) Refine(C_0, b): $A = \{q_3, q_5\}$. 1 splits.



Colour:	states:
0	q_1, q_6
1	q_2, q_4, q_7
2	q_0
3	q_3, q_5

(c) Refine(C_3, a): $A = \{q_1, q_4, q_6\}$. 1 splits.



Colour:	states:
0	q_1, q_6
1	q_2, q_7
2	q_0
3	q_3, q_5
4	q_4

(d) The partition is now *stable*.

Figure 4.2: Example of Algorithm 5.

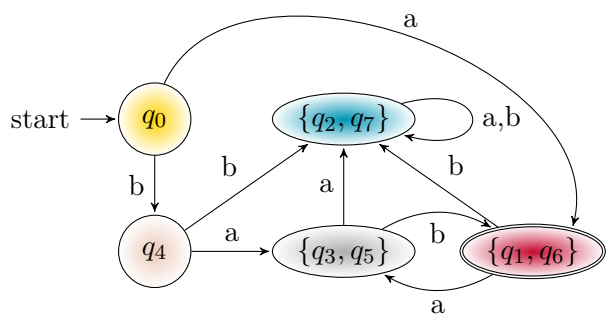
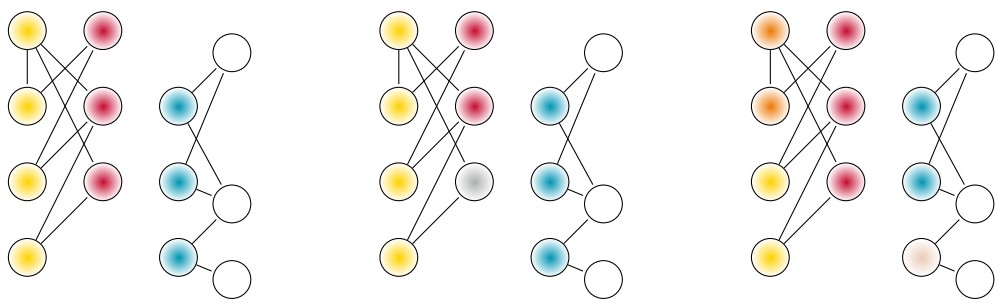


Figure 4.3: Minimised Example DFA (see Figure 4.1)



(a) π is stable for Red (b) π' is stable for Red $\cup$ Grey (c) π' is stable for Red

Figure 4.4: Visual behind Observation 4.8

Lemma 4.9 (Refining a Stable Colour Class)**.** *Let $C \subseteq Q$ be partitioned into $C_0, \ldots, C_{\ell-1}$. For any $a \in \{0, \ldots, \ell-1\}$: If a partition π is stable for C and for every $b \in \{0, \ldots, \ell-1\} \setminus \{a\}$, π is stable for C_b. Then π is stable for C_a as well.*

Proof. Suppose that the contrary is true: π is not stable for C_a.
This implies that there exists q_i, q_j and x, where $q_i \equiv_\pi q_j$ such that (w.l.o.g.) $\delta(q_i, x) \in C_a$, but $\delta(q_j, x) \notin C_a$. Let us define $q_i' = \delta(q_i, x)$ and $q_j' = \delta(q_j, x)$.
By construction $q_i' \in C_a \subseteq C$, which implies that $q_i' \in C$.
Since π is stable for C and $q_i' \in C$, it means that $q_i', q_j' \in C$.
Therefore there exists a $b \neq a$ such that $q_j' \in C_b$ and $q_i' \notin C_b$.
This implies that π is not stable for C_b, which contradicts that π was stable for all C_i except a. Therefore π must be stable for C_a. $\square$

Figure to be added

Figure 4.5: Visualisation of Lemma 4.9

Obtaining the speed-up.

Suppose we refine partition π to π' with Colour C_i and this operation splits the colour class C_j into several new classes. Lemma 4.9 lets us skip refining with (the largest) one of these new classes if π' is stable for C_j. In particular, this is the case, if π is stable for C_j or if $C_j = C_i$. Since, if we refine with all the other classes created from C_j, the resulting partition is stable for each of them and, by Lemma 4.9, so is the remaining one. Therefore, checking all classes except the largest newly created class allows us to skip a big chunk of the calculations.

4.3.2 Fast DFA Minimisation Algorithm

Lemma 4.9 allows us to speed up our basic algorithm. Algorithm 6 improves the basic algorithm and also terminates with a stable colouring.

Algorithm 6 Fast DFA Minimisation Algorithm

```
Init: Create the partition π = {F, Q \ F} = {C_0, C_1}
Set Queue = [0]

while Queue ≠ ∅ do
    Take a C from the Queue

    Execute Refine(C)
    if Refine(C) splits C_i into two or more new classes then
        Find a set ℓ, ..., k of the correct number of unused colours.
        Call one class C_i and the others C_ℓ, ..., C_k.
        if i ∈ Queue then
            Add C_ℓ, ..., C_k to Queue
        else
            Find ℓ* = arg max_{j∈{i,ℓ,...,k}} {|C_j|}
            Add all colours except ℓ* to Queue
```

An Example

In Figure 4.6, we apply Algorithm 6 on the same DFA as in Figure 4.1. Note that as defined before, Refine(C_0) means applying both Refine(C_0, a) and Refine(C_0, b). As mentioned before, Refine(C_i, a) can be interpreted as that we

refine all colours based on the number of transition with label a **into** colour C_i.

Theorem 4.10. *Algorithm 6 terminates with a partition into equivalent states.*

Proof. Very similar to the proof of Theorem 4.7 and left as an exercise. □

4.3.3 The Complexity of the Algorithm

In this section, we show that the partition into equivalent states using Algorithm 6 can be computed in $O(m \log n)$. Here n denotes the number of states and m is the total number of transitions in the DFA.

To do that, we first prove two Lemmas. The first Lemma shows how often we visit each state. The second one shows the time complexity of Refine(C).

Lemma 4.11. *In Algorithm 6, every state is used at most* $\log_2 n$ *times in a refining operation, where* n *denotes the number of states in the DFA.*

Proof. Consider a state q.

Let $C_0, C_1, \ldots, C_\ell$ be all different colour classes that contain q, that have been used in refining operations. So $\{q\} \subseteq C_\ell \subseteq C_{\ell-1} \subseteq \ldots \subseteq C_0$.

Then because of our queuing rule: For every i, C_{i+1} is obtained from splitting up C_i into at least two parts, and C_{i+1} is the smaller part, so $2|C_{i+1}| \leq |C_i|$.

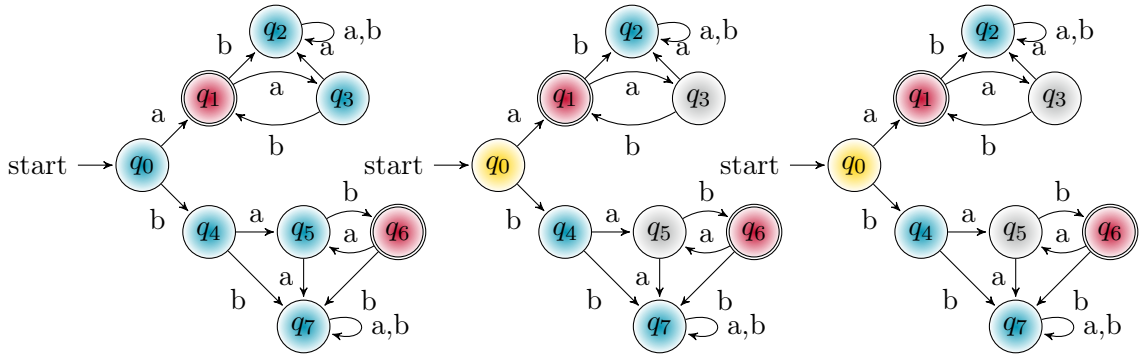
It follows that $|C_0| \geq 2^\ell$. Obviously $|C_0| \leq n$, so $\ell \leq \log_2 n$. □

Definition 4.12. For a state set C, we denote all transitions into C by $E^-(C)$.

Lemma 4.13. *Algorithm 6 can be implemented such that the operation* Refine(C) *takes time* $O\big(|E^-(C)|\big)$.

Proof. The efficient implementation of Algorithm 6 mostly requires careful use of data structures. We keep track of the following items.

- For each state q and symbol x, a list $N_x^-(q)$ of all states q' with $\delta(q', x) = q$, i.e., all incoming x-transitions. Similarly, for each $C \subseteq Q$ we denote by $N_x^-(C) = \bigcup_{q \in C} N_x^-(q)$, all incoming x-transitions into the set C.
- Queue A list that contains the cells (colours/numbers) that need to be considered for refining operations.
- For each cell $i \in \{0, \ldots, k-1\}$:
  - C_i, a *DLL* containing all states of C_i.



Colour:	states:
0	q_1, q_6
1	$q_0, q_2, q_3, q_4, q_5, q_7$

QUEUE: 0

(a) Initial Colouring

Colour:	states:
0	q_1, q_6
1	q_2, q_4, q_7
2	q_0
3	q_3, q_5

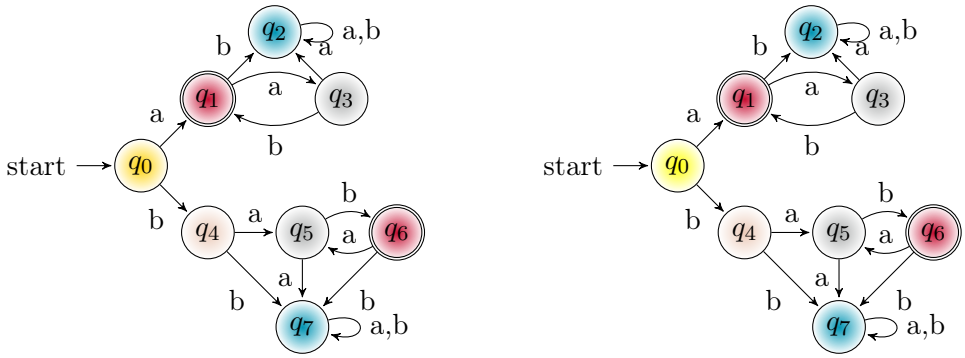
QUEUE: ~~0~~,2,3

(b) After REFINE(C_0).

Colour:	states:
0	q_1, q_6
1	q_2, q_4, q_7
2	q_0
3	q_3, q_5

QUEUE: ~~0~~, ~~2~~, 3

(c) After REFINE(C_2)



Colour:	states:
0	q_1, q_6
1	q_2, q_7
2	q_0
3	q_3, q_5
4	q_4

QUEUE: ~~0~~, ~~2~~, ~~3~~, 4

(d) After REFINE(C_3)

Colour:	states:
0	q_1, q_6
1	q_2, q_7
2	q_0
3	q_3, q_5
4	q_4

QUEUE: ~~0~~, ~~2~~, ~~3~~, ~~4~~

(e) After REFINE(C_4)

Figure 4.6: Fast DFA minimisation example.

  - InQueue[i] A boolean that indicates if $C_i \in$ Queue.

- For each state $q \in Q$:
  - A pointer to the DLL item that represents q.
  - Colour[q] The current cell of q.

A DLL is a doubly linked list, of which the methods that we need have exactly the runtime complexity necessary to make our proof work (see Lemma 4.14.

We prove the lemma statement for Refine(C, x), where we replace $E^-(C)$ by $E^-(C, x)$, the incoming x-transitions into C. Note that this is sufficient as the lemma statement then follows from summing over all $x \in \Sigma$. Thus we prove Refine(C, x) runs in time $O(t)$, with $t = |E^-(C, x)|$. Using the above data structures, we compute the refinement as follows.

- Loop over all $q \in Q$ and $q' \in N_x^-(q, x)$ in time $O(t)$, and compute:
  - L, a list of all cells i such that C_i contains a $q' \in N_x^-(C)$,
  - $A[i] = |N_x^-(C) \cap C_i$, the number of such states in C_i.
- In time $O(|L|) \leq O(t)$, loop over L and compute:
  - For each $i \in L$: if $A[i] < |C_i|$. In this case cell i splits up into two new cells.
  - If so, choose a new colour ℓ and update Queue by adding either i or ℓ (according to the rule implied by Lemma 4.9.
- In time $O(t)$, loop over all $q \in Q$ and $q' \in N_x^-(v)$, and:
  - Update the colours of the states (update the DLLs and colour[q']). Moving a state from one DLL to another takes $O(1)$ time (see Lemma 4.14).

Verifying that, indeed, the running times in the above mentioned computations can be maintained is straighforward and completes the proof. □

We have shown that it is possible to bound the complexity of our Algorithm by $O(m \log n)$. To achieve this claimed computation bound in practice, there are some requirements for our data structures. An implementation should satisfy the following criteria:

1. Accessing the class to which a state belongs in constant time.
2. Enumeration of the elements of a class in time proportional to its size.

3. Adding and removing an element in a class in constant time.

A structure that satisfies all these criteria is a Doubly Linked List.

Lemma 4.14 (Complexity DLL methods)**.** *For a DLL of length n:*

- *All elements can be listed in time $O(n)$.*
- *A single DLL item can be deleted in time $O(1)$.*
- *An element can be added to the start or end of the list in time $O(1)$.*

While a DLL allow us to prove that Algorithm 6 runs in $O(m \log n)$ operations theoretically, in practice, using DLLs is not an efficient implementation in Python. This is mainly caused by the fact that Python has not build-in DLL data structure, making other, build-in data structures a better choices. Python sets are a built-in structure that allow for the same computation time complexity, yet for average case computation times instead of worst case. That said, the time advantage gained from using a build-in data structure compaired to a self-implemented one far outweighs the disadvantage of worst-case versus average-case performance. Of course, one can experiment with other date structures, and the exact way to implement that structure is left to the reader.

Theorem 4.15 (The Complexity of Algorithm 6)**.** *For any DFA $D = (Q, \sum, \delta, q_0, F)$, the partition into equivalent states can be computed in time $O(m \log n)$, where n is the number of states and $m \leq n|\sum|$ is the total number of transitions.*

Proof. The complexity of Algorithm 6 is dominated by the complexity of the refining operations. Therefore we ignore the various initialisation steps.

From Lemma 4.13, we know that Refine(C) has a complexity $O\big(|E^-(C)|\big)$. We sum over all refining operations that take place during the whole algorithm

$$\sum_C O\Big(\big|E^-(C)\big|\Big)\,. \tag{4.1}$$

Here, the sum over all C's denotes all the different C's with which we refine the partition in some step of the algorithm.

This sum can be rewritten into a sum over all states. When doing so, the runtime has to be multiplied by the number of times each state has been used in the refinement operation. Doing this, allows us to rewrite (4.1) into

$$\sum_{q \in Q} O\Big(\big|E^-\big(\{q\}\big)\big|\Big)\,(|\{\, C \mid q \in C \text{ and } \textsc{Refine}(C) \text{ is used during the algorithm}\,\}|)\ . \tag{4.2}$$

From Lemma 4.11, we know that the number of times a state is used in a refinement operation is upper bounded by $\log_2 n$. Where n denotes the total amount of states in the DFA. This allows us to upper bound our sum by:

$$\sum_{q \in Q} O\Big(\big|E^-(\{q\})\big|\Big) \cdot \log_2 n$$

Note that the sum of all neighbours of all states is just the amount of transitions. Therefore changing to $E^-(Q)$, which also denotes that, does not change our expression. Finally, we are then also able to rewrite it into the desired order. Here m denotes the total number of transitions.

$$O\Big(\big|E^-(Q)\big|\Big) \cdot \log_2 n = O(m \log n) \qquad \square$$

4.4 Application on the GI/#Aut Problems

We have discussed an algorithm that minimises DFAs by partitioning into equivalent states. Finally, we are turning towards our original problem: The Graph Isomorphism Problem. Let us recall the invariant, the Refine() operation and the complexity theorem. The corresponding proofs can be found with the original statements.

Invariant 4.3. If $q_i \not\equiv_\pi q_j$, then there exists a word w that distinguishes q_i from q_j.

Lemma 4.5 (Refine(C, x) operation for DFA)**.** Let C be a colour class in the current partition. Let $x \in \sum$ be a symbol. Let $A \subseteq Q$ be all states q_i with $\delta(q_i, x) \in C$. Then replacing each cell D in the current partition by $D_A = D \cap A$ and $D_{\overline{A}} = D \setminus A$ provided both are nonempty, maintains the invariant.

Theorem 4.15 (The Complexity of Algorithm 6)**.** For any DFA $D = (Q, \sum, \delta, q_0, F)$, the partition into equivalent states can be computed in time $O(m \log n)$, where $n = |Q|$ and $m \leq n|\sum|$ is the total number of transitions.

Notice that in the DFA problem, we use the symbols in the alphabet of the DFA. In the GI-problem, we don't have such symbols. Therefore, we are going to define the labels of our "transitions" differently. Remember from Chapter 2 that a colour in a graph says something about its neighbourhood. Instead of using symbols with which a state transitions into a colour class to distinguish them, we use the number of neighbours a vertex has in colour class C. Hence, if we refine using colour C, we partition all colour classes based on the number of neighbours of colour C that a vertex has.

4.4.1 The Refining Operation

The goal of our algorithm is to find the coarsest stable partition. Since we are refining a partition into this, we want our partition to always be able to be refined into it. Thus, we make it so that the coarsest stable partition is always a refinement of our current partition.

Invariant 4.16 (The GI Invariant)**.** *For any two vertices x and y, if x and y have distinct colours in any iteration, then x and y have distinct colours in the coarsest stable partition.*

Lemma 4.17 (Refine(C, i) operation for GI)**.** *Let C be a colour class in the current partition. Let $i \geq 0$ be an integer. Let A denote the set of all vertices with exactly i neighbours in the colour class C. Split all colour classes (D) in the current partition as follows: $D_A = D \cap A$ and $D_{\bar{A}} = D \setminus A$, provided both are non-empty. Then this operation maintains the invariant.*

Proof. TBA (similar to Lemma 4.5) □

Theorem 4.18 (The Complexity of fast colour refinement for the GI-problem)**.** *For any graph, the partition into equivalent states can be computed in time $O(m \log n)$, where $n = |Q|$ and $m \leq n|\sum|$ is the total number of transitions.*

Proof. TBA (similar to Theorem 4.15) □

4.4.2 The Starting Point

In Algorithm 6, we started with the partition of accepting and non-accepting states. Since the vertices in a graph do not have any underlying difference, we have to look for a different way to partition our vertices. One straightforward solution would be to start with a uniform colouring. Of course, when using the algorithm during branching, we would apply it to $\alpha(D, I)$.

4.4.3 Be aware!

There are some important things to keep in mind while implementing Algorithm 6 for the Graph Isomorphism problem.

- Make sure that the data structures that you use and the complexity of their methods fit your need;
- Crucial for your running time is that every element from lemmas 4.9 and 4.13 is properly implemented. This means (among other things) that

- Refine(C) should run in $O(t)$, with t the number of edges incident with a vertex in C. To make this work, you have to be really careful with how you loop over the vertices of the graph. E.g., generally, you cannot loop over all vertices of the graph, nor can you loop over all neighbours of all vertices in $N(C)$.
- You need to have the correct rule to put a colour class into Queue.

Chapter 5

Automorphism Groups

5.1 Introduction

One of our colour refinement algorithms combined with the branching algorithm solves both the GI and the #Aut problem. Using this method when counting the number of automorphisms of graph G, we generate each automorphism at one leaf of the branching algorithm. Doing this for graphs with a large number of automorphisms is an issue. Remember that a graph with n vertices may have up to $n!$ automorphisms (e.g., the complete graph on n vertices or the graph with n vertices but no edges). This grows exponentially in n For example, bigtrees3.grl is a tree with 227 vertices. This graph has 2 772 351 862 699 137 701 073 289 910 157 312 automorphisms. The age of the universe in milliseconds is 430.000.000.000.000.000.000. So even if we started computations at the inception of the universe and had generated an automorphism every millisecond, today, we would still be nowhere near finishing our count. Therefore, our current algorithm is not a feasible approach to solve the #Aut problem. This Chapter is about a smarter method, using some tools from Algebra (the permutation group, generating sets, orbits, stabilisers and transversals) to reduce the number of automorphisms that we generate. When implemented, this results in a (significantly) smaller branching tree and therefore a much smaller running time. We note here, that this only leads to improvements for the #Aut problem, not for the GI problem.

5.1.1 Permutations to Generate Automorphisms

We explain some of the ideas behind the reduction of the branching tree for the #Aut problem with the following example. Consider the graph C_6, a cycle with 6 vertices (see Figure 5.1).

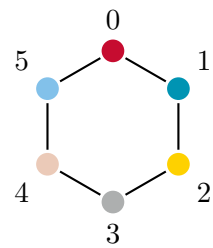


Figure 5.1: Coloured C_6.

The key idea is to look differently at the #Aut problem. Instead of asking "How many automorphisms can we create for C_6?", we ask ourselves "How can we generate automorphisms of C_6?"

The cycle C_6 can be transformed in two ways that maintain the same general shape: rotation and mirroring. We rotate it clockwise by 60° and mirror it along the vertical axis. We consider this transformation as a permutation (a mapping) of the set of vertices of C_6 onto itself ($V(C_6) \to V(C_6)$). That is, one rotation maps vertex $i \to i + 1 \mod 6$ for all $i \in \{0, \ldots, 5\}$. If we apply this rotation six times, each vertex is mapped onto itself (the trivial permutation). Thus, we can generate six automorphisms only using rotation. When mirroring, we reach the trivial permutation after applying the same transformation twice. Therefore, we can generate two automorphisms using mirroring (mirrored and not mirrored). All automorphisms that can be generated by using both mirroring and rotation are shown in Figure 5.2.

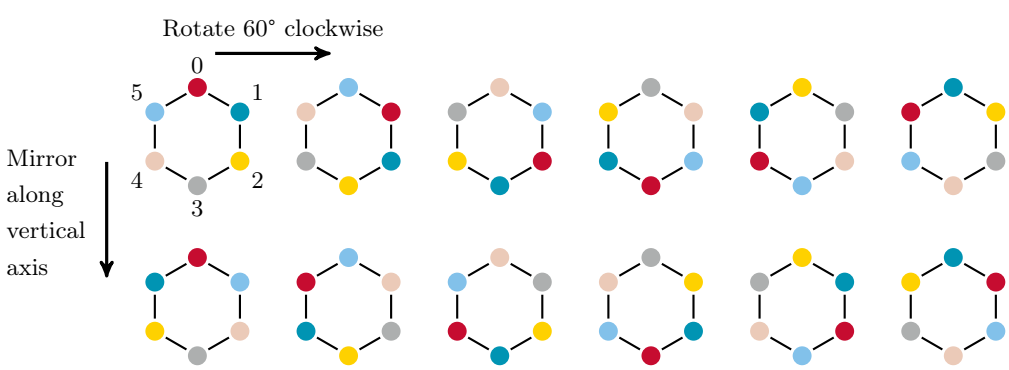


Figure 5.2: Automorphisms of C_6 using Rotation and Mirroring

It turns out that these are indeed all twelve automorphisms of C_6. All twelve of them can be found using only two operations. That will be the idea of this chapter: finding a set operations (or permutations) that generate all automorphisms and

then computing the number of (unique) automorphisms that they generate.

Two things remain to be done: First, implement this idea more formally and precisely. This is done using a generator for $\mathrm{Aut}(G)$, the group of all automorphisms of G under function composition, using theory about permutation groups. Second, to check if and how this improves our algorithm.

The general idea is that, if each automorphism group has a representation by a small set of automorphisms, and from such a representation we can compute the number of automorphisms that a graph has, we can use this to (hopefully) create an efficient algorithm to compute the number of automorphisms, even for graphs that have many.

5.1.2 Permutation Groups

A permutation is a bijection (an invertible mapping, a function that is both one-to-one and onto) from a set S onto itself: $\sigma : S \to S$. Recall the definition of a permutation group.

Definition 5.1 (Permutation Groups [3])**.** In general, the permutations of a set X form a group S_X. If X is a finite set, we can assume $X = \{1, 2, \ldots, n\}$. In this case we write S_n instead of S_X. We call S_n *the symmetric group on n elements.* A subgroup of S_n is called a *permutation group.*

Since permutations can also be seen as functions, we adopt the convention from [3] that for any two permutations $p, q \in S_n$, we have $pq(x) = (p \circ q)(x) = p(q(x))$, i.e., right multiplication of p with q in S_n *first* applies q and *then* applies p.

Theorem 5.2. *The symmetric group on n letters, S_n, is a group with $n!$ elements, where the binary operation is function composition.*

Proof. See Theorem 3.3.1 from the Algebra reader [3]. □

Since an automorphism is a permutation of the vertices of a graph we can represent the operations from Section 5.1.1 on C_6 as permutations. A way to do this is shown in Figure 5.3.

In particular, we can generate all these automorphisms with the two permutations $p = (0\ 1\ 2\ 3\ 4\ 5)$ and $q = (1\ 5)(2\ 4)$. For example, applying q after p generates the second graph on the second row. This is equivalent to applying the (function) composition of q and p.

$$q \circ p = (0\ 5)(1\ 4)(2\ 3)\,. \tag{5.1}$$

As we see in Figure 5.3, there are multiple ways generate that same automorphism with only p and q. In this case, we can also apply $p^{-1} = p^5 = p \circ p \circ p \circ p \circ p$

Permutation	Operation	Mapping	Cycle Notation
p	Rotate 60° cw	$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 1 & 2 & 3 & 4 & 5 & 0 \end{pmatrix}$	$(0\ 1\ 2\ 3\ 4\ 5)$
q	mirror vertically	$\begin{pmatrix} 0 & 1 & 2 & 3 & 4 & 5 \\ 0 & 5 & 4 & 3 & 2 & 1 \end{pmatrix}$	$(1\ 5)(2\ 4)$

Figure 5.3: Automorphisms of C_6 using Permutations

after q. Note that once we've mirrored vertically, going left to right in Figure 5.1 corresponds to counterclockwise rotation. Therefore, we have

$$p^{-1} \circ q = (0,5)(1,4)(2,3)\,. \tag{5.2}$$

There are now two things that are important to note. These are written below.

Observation 5.3 (Inverse)**.** *If p is an automorphism of G, then p^{-1} is also an automorphism of G.*

Proof. Suppose p is an automorphism of G. Define p^{-1}, to be the permutation such that $p^{-1}(v) = u \iff p(u) = v$. Note that p^{-1} is well-defined since p is a bijection. Then $p^{-1} \circ p = ()$, where $()$ is the Trivial automorphism, since $p^{-1}(p(v)) = v$ for all $v \in V$. Now, for some $u, v \in V$ let $u' = p^{-1}(u)$, $v' = p^{-1}(v)$. Then $u = p(u')$ and $v = p(v')$ and, since p is an automorphism of G, $\{u', v'\} \in E \iff \{p(u'), p(v')\} \in E$. Therefore, equivalently $\{p^{-1}(u), p^{-1}(v)\} \in E \iff \{u, v\} \in E$. □

Observation 5.4 (Composition)**.** *If p and q are automorphisms of G, then $q \circ p$ is also an automorphism of G.*

Proof. Since p and q are both automorphisms of G, we have $\forall u, v \in V : \{u, v\} \in E \iff \{p(u),\, p(v)\} \in E \iff \{q(u),\, q(v)\} \in E$. For some $u, v \in V$, let $u' = p(u)$ and $v' = p(v)$. Then, $\{u', v'\} \in E \iff \{q(u'),\, q(v')\}$. Thus, $\forall u, v \in V : (u, v) \in E \iff \{p(u),\, p(v)\} \in E \iff \{q(p(u)),\, q(p(v))\}$. □

These observations may lead the reader to believe that the automorphisms of G form a group. Indeed they do!

Theorem 5.5 (Aut(G)). *The automorphisms of G form a group under composition.*

Proof. To show this, we need to show closure, the existence of an identity and inverse, and associativity.
Closure: By Observation 5.4, $\forall\, p, q \in \mathrm{Aut}(G) : q \circ p \in \mathrm{Aut}(G)$.
Identity: The *Trivial automorphism:* () satisfies $\forall\, p \in \mathrm{Aut}(G) : p \circ () = () \circ p = p$.
Inverse: By Observation 5.3 $\forall\, p \in \mathrm{Aut}(G) : \exists\, p^{-1} \in \mathrm{Aut}(G)$ such that $p^{-1} \circ p = p \circ p^{-1} = ()$.
Associativity: $\forall\, p, q, r \in \mathrm{Aut}(G) : (r \circ q) \circ p = r \circ (q \circ p)$. This follows from $((r \circ q) \circ p)(v) = (r \circ q)(p(v)) = r(q(p(v))) = r((q \circ p)(v)) = r \circ (q \circ p)$. □

From here on, we write pq instead of $p \circ q$.

Important to remember is that automorphisms (and permutations) are *not commutative* in general:

$$pq \neq qp \qquad \text{(in general)}\,.$$

Since each automorphism is a permutation of the vertices $V = V(G)$ of a graph, the automorphism group $\mathrm{Aut}(G)$ is a subgroup of the Symmetric group on V. We denote this group by $\mathrm{Sym}(V)$.

5.2 Generating Sets

A generating set of a subgroup H of $Sym(V)$ is a set of permutations on V, such that every automorphism from H is equal to the composition of some number of copies of permutations from H in any order.

Definition 5.6 (Generating Sets). Given a set of permutations $\{p_1, \ldots, p_k\}$ on V, by $\langle p_1, \ldots, p_k \rangle$ we denote the smallest subgroup H of $Sym(V)$ that includes all these permutations. $\{p_1, \ldots, p_k\}$ is called a generating set of H.

In the example from Figure 5.1, the automorphism group is generated by a set of two permutations

$$\mathrm{Aut}(G) = \langle p, q \rangle = \langle (0\ 1\ 2\ 3\ 4\ 5), (1\ 5)(2\ 4) \rangle\,.$$

Figure 5.4 shows how p (red) and q (blue) generate $\mathrm{Aut}(G)$.

Note that generating sets do not have to be of the same size. Let us say we take p, p^{-1}, q, then we have a generating set. However, eliminating either p or p^{-1} leaves

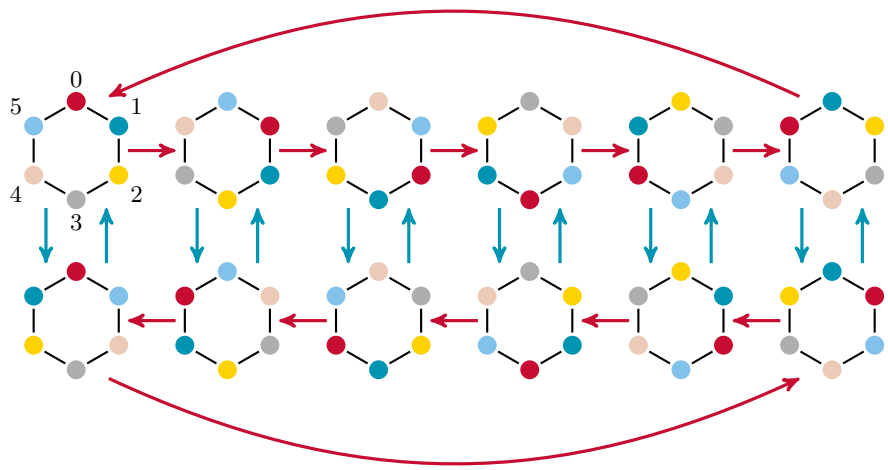


Figure 5.4: Using p and q to Generate $\mathrm{Aut}(G)$ (p transformations in red; q transformations in blue).

us with a smaller generating set. This also immediately shows that the smallest sets are not unique. Since $\langle p^{-1}, q\rangle$ and $\langle q^{-1}, p\rangle$ both generate $\mathrm{Aut}(G)$.

Let us now look at a second example: a complete graph with n vertices (K_n). Let $V(G) = V = \{0, \ldots, n-1\}$. Since this is a complete graph, $Aut(K_n) = Sym(V)$.

Proposition 5.7. $Aut(K_n) = \langle (0\ 1\ \ldots\ n-1), (0\ 1)\rangle$.

Proof. Let $p = (0\ 1\ \ldots\ n-1)$ and $q = (0\ 1)$. First note that it suffices to show that we can generate all permutations of the form $(0\ 1\ \ldots\ i-1\ i+1\ i\ i+2\ \ldots\ n-1)$, a swap of two arbitrary consecutive elements. We claim that

$$p^i q p^{-i}$$

generates the permutation that swaps (only) elements i and $i+1$. Indeed, for all $v \in \{0, \ldots, n-1\}$

$$\begin{aligned} p^i q p^{-i}(v) &= p^i q(v-i) \\ &= \begin{cases} p^i(v-i) & \text{if } (v-i) \notin \{0,1\} \\ p^i(v-i+1) & \text{if } v-i = 0 \\ p^i(v-i-1) & \text{if } v-i = 1 \end{cases} \\ &= \begin{cases} v & \text{if } v \notin \{i, i+1\} \\ i+1 & \text{if } v = i \\ i & \text{if } v = i+1 \end{cases} . \end{aligned}$$

□

This example shows that there are graphs with large automorphism groups that can be generated with a small generating set. In this particular case, we generated a set of $n!$ elements with only two permutations.

Recall that this is indeed what we look for. Yet, to be able to use this in efficient computations, we need this to be consistently true. Every automorphism group should have a small generating set, relative to the size of the graph.

5.3 Polynomial Size Generating Sets

In this section, we investigate how to iteratively create a generating set for the automorphism group of a graph G. To this end, consider the branching algorithm (Algorithm 2). Using that algorithm, we can find all automorphisms of G. Namely, instead of only adding to a counter when we find an automorphisms at a leaf of the branching tree, we can also store the automorphism that we find in a set X. Doing that, we obtain a set that contains all automorphisms in $\mathrm{Aut}(G)$, which is a generating set of $\mathrm{Aut}(G)$ by definition. Algorithm 7 together with Algorithm 8 is a pseudo-code version of this idea.

Algorithm 7 CalculateAutomorphisms(G)

Input: Graph $G = (V, E)$
Action: Calculate generating set $\langle X \rangle$ of $Aut(G)$ (of polynomial size)
Output: Size of $Aut(G)$

Global $X := \emptyset$
SimpleGeneratingSet$\big([\,], [\,]\big)$

Return Order(X)

The recursion step is similar to Algorithm 2. Instead of just checking whether the colouring defines a bijection, it also checks if the resulting automorphism is generated by $\langle X \rangle$. If not, then the bijection is added to X. Otherwise, it is not added. This check is called Membership testing and is further discussed in Section 5.5.3. The reason to do this additional check, is to keep the resulting generating set small.

When the colouring is not yet a bijection, we need to branch. Note that the chosen C and x are called branching cell and vertex, respectively. The recursion algorithm pseudocode is given in Algorithm 8.

Algorithm 8 SimpleGeneratingSet(D, I)

Input: D and I are equal length sequences of vertices of G
Action: Ensures that $\langle X \rangle$ contains all automorphisms f that follow D, I

ColourRefinement$(\{G, G\})$ with starting colouring $\alpha(D, I)$
if there is no automorphism following $\alpha(D, I)$ **then** Return
if there is unique automorphism f following $\alpha(D, I)$ **then**
  if $f \notin \langle X \rangle$ **then**
    $X := X \cup \{f\}$
  Return

Choose a branching cell C with $|C| \geq 2$
Choose a branching vertex $x \in C$
for $y \in C$ **do**
  SimpleGeneratingSet$(D + x, I + y)$

Correctness

Lemma 5.8 (Correctness of Algorithm 7)**.** *The recursive calling of SimpleGeneratingSet ensures that $\langle X \rangle$ contains all automorphisms that follow D, I.*

Proof. Left as an exercise. It is done by induction over the recursion tree. The base case is when either there is no automorphism following $\alpha(D, I)$, or there is unique automorphism f following $\alpha(D, I)$. □

For all of the above to be of any use to us, we need the generating set of the automorphism to be small. Moreover, this has to hold for all automorphism groups. If this is not the case, and, e.g., there are graphs for which the automorphism group has only generating sets that are roughly as large as the group itself, and that group is very large, then Algorithm 7 still runs for a very long time, since each permutation in the generating set needs to be found.

So far, we have seen some evidence that automorphism groups indeed have small generating sets (all automorphism groups that we considered had small generating sets independent of the order of the group), but the following observation helps us to give a general upper bound.

Observation 5.9. *Let $H_1 = \langle p_1, \ldots, p_k \rangle, q \notin H_1$, and $H_2 = \langle p_1, \ldots, p_k, q \rangle$. Then $|H_2| \geq 2|H_1|$.*

Proof. By Lagrange's Theorem [3, Theorem 5.2.3], $|H_1|$ divides $|H_2|$. Since $q \in H_2$ and $q \notin H_1$, we know that $|H_1| \neq |H_2|$, so $|H_2| \geq 2|H_1|$. □

This observation directly implies that k, the size of a generating set, can be bounded.

Lemma 5.10 (Small Generating Sets)**.** *Given G with $|V| = n$, and H a subgroup of $Sym(V)$. Then a generating set $\{p_1, \ldots, p_k\}$ of H s.t. $\forall\ i,\ p_i \notin \langle p_1, \ldots, p_{i-1}\rangle$ has polynomial size: $O(n \log n)$.*

Proof. Since for all $p_i \in \langle p_1, \ldots, p_k\rangle$ it holds that $p_i \notin \langle p_1, \ldots, p_{i-1}\rangle$, Observation 5.9 implies that $|H| = 2^k$. Furthermore, $|H| \leq n!$ implies that $k \leq \log_2 |H| \leq \log_2 n! \leq O(n \log n)$. □

We now know that every automorphism group has a small generating set. That does not mean that every generating set of an automorphism group is small! Recall that the set that contains all automorphisms is also a generating set of the automorphism group. The approach we described in this section, does find a small generating set. However, we still compute all automorphisms and check for each one if it needs to be added. Therefore, we might as well counted them to get our answer. Thus, we need to add one more step. We want to be able to predict for a significant number of branches in the branching tree if they will lead to an automorphism that is already generated by the set that we currently have. If turns out that we can do this.

5.4 Pruning the Branching Tree

Recall from Section 3.3 that the number of leaves in the recursion tree is always at least the number of automorphisms of G. This section shows how to reduce the number of visited leaves through symmetry and maintaining a set of automorphisms that in the end will generate $Aut(G)$.

Example

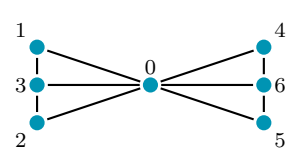


Figure 5.5: Example Graph

For the graph G shown in Figure 5.5, the complete branching tree (with corresponding permutations) is shown in Figure 5.6. The branching tree is written down as

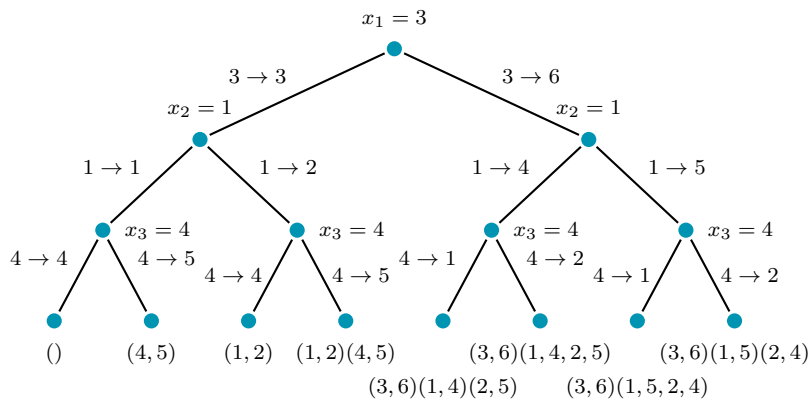


Figure 5.6: Branching Tree of Figure 5.5.

follows: The nodes denote a choice of a branching vertex and the edges represent what that vertex is being mapped to. I.e., the top node represents vertex 3, which is being to 3 and 6 on the left and right edge respectively.

Each leaf of this tree specifies a permutation, i.e., automorphism. Algorithm 7 produces a generating set consisting of three permutations

$$Aut(G) = \left\langle (4,5),\ (1,2),\ (3,6)(1,4)(2,5) \right\rangle .$$

Ideally, we do not spend any computations on the other permutations. If we can predict which branches lead to permutations that our set already generates, we greatly reduce the necessary calls on our colour refinement algorithm.

To show the significance of this, Figure 5.7 shows the branching tree of our previous example, but with all the nodes that do not need to be considered greyed out.

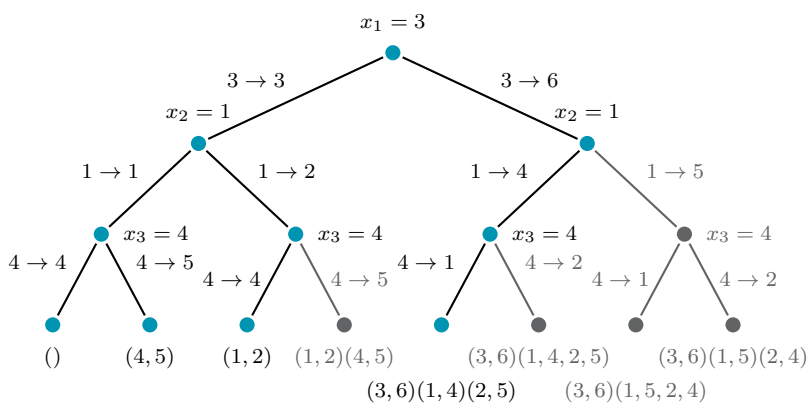


Figure 5.7: Branching Tree of Figure 5.5 with unnecessary branches greyed out.

The following lemma tells us that we can, indeed, predict that exactly the greyed out branches in Figure 5.7 can be ignored.

Lemma 5.11 (Pruning the Branching Tree)**.** *Let $\{x_1, \ldots, x_k, y\} \subseteq V(G)$. If a subgroup H of $Aut(G)$ contains all automorphisms p with $p(x_i) = x_i$ for all i, and at least one automorphism q with $q(x_i) = x_i$ for all $i \leq k-1$ and $q(x_k) = y$, then H contains all automorphisms of G of the latter type.*

Proof. Choose any automorphism $r \in Aut(G)$ with $r(x_i) = x_i$ for all $i \leq k-1$ and $r(x_k) = y$. Then $s := q^{-1}r$ satisfies $s(x_i) = x_i$ for all $i = 1, \ldots, k$, so $s \in H$. Therefore, $r = qq^{-1}r = qs \in H$. □

Let us take this lemma apart to better understand it. Starting with the automorphism subgroup H of $Aut(G)$. This group is constructed such that it contains all automorphisms that leave a sequence of k vertices unchanged ($\{x_1, \ldots, x_k\}$). Furthermore, there also exists an automorphism $q \in H$ that does not preserve the whole sequence, but only up to $k-1$ ($\{x_1, \ldots, x_{k-1}\}$). For this q, x_k is mapped onto a different vertex $y \notin \{x_1, \ldots, x_k\}$. If this is satisfied, then H does not only contain all automorphisms that leave $\{x_1, \ldots, x_k\}$ unchanged, but also all automorphisms that leave $\{x_1, \ldots, x_{k-1}\}$ unchanged and maps x_k to y.

This means that once all automorphisms that follow $(D + x, D + x)$ have been found, as well as a single automorphism following $(D + x, D + y)$, the set of found automorphisms generates all other automorphisms that follow $(D + x, D + y)$ as well. Therefore, we can stop exploring this branch and continue from the node corresponding to (D, D).

Example

To illustrate the pruning that Lemma 5.11 allows, Figure 5.8 shows the branching tree with all pruned nodes greyed out and all trivial nodes and mappings in red.

Since the trivial permutation is always an automorphism, it is always an element of $\mathrm{Aut}(G)$ (otherwise $\mathrm{Aut}(G)$ would not be a group). Consider the path that leads us there (in this case), $3 \to 3,\ 1 \to 1,\ 4 \to 4$. In the notation of Lemma 5.11, we have $\{x_1, x_2, x_3\} = \{3, 1, 4\} \subseteq V(G)$ and subgroup $H = \langle\rangle$ of $Aut(G)$.

Now take one step back in this path (here $x_3 = 4$). Note that only $4 \to 5$ also leads to an automorphism $q = (4, 5)$. In the notation of Lemma 5.11, we have $\{x_1, x_2, x_3, y\} = \{3, 1, 4, 5\} \subseteq V(G)$ and subgroup $H = \langle(4, 5)\rangle$ of $Aut(G)$. Thus, H contains all automorphisms of G that fix $\{3, 1\}$ and maps 4 onto 5. Since this was the only non-trivial mapping that led to an automorphism, H contains all automorphisms that fix $\{3, 1\}$.

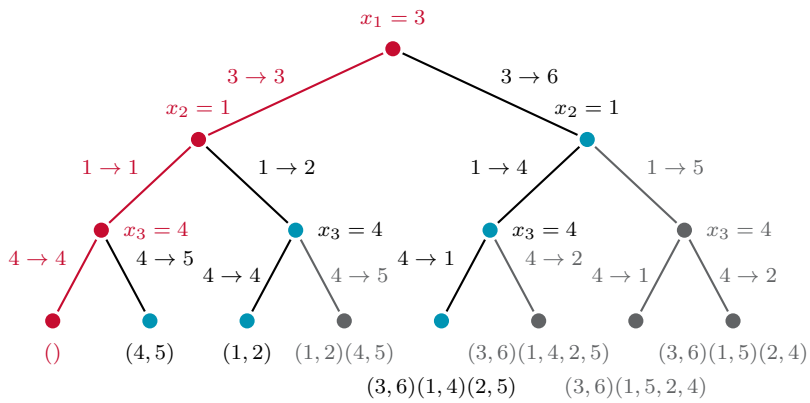


Figure 5.8: Pruned Branching Tree of Figure 5.5 with the complete trivial path in red.

Now we take another step back on the trivial path (here $x_2 = 1$). Note that only $1 \to 2$ also leads to automorphisms. Let us find one: $q = (1,2)$. In the notation of Lemma 5.11, we have $\{x_1, x_2, y\} = \{3,1,2\} \subseteq V(G)$ and subgroup $H = \langle (4,5),\ (1,2) \rangle$ of $Aut(G)$. Therefore, H contains all automorphisms of G that fix $\{3\}$ and maps 1 onto 2. Since this was the only non-trivial mapping that led to an automorphism, H contains all automorphisms that fix $\{3\}$.

Finally, we take our last step back on the path (here $x_1 = 3$). Note that only $3 \to 6$ also leads to automorphisms. Let us find one: $q = (3,6)(1,4)(2,5)$. *Note that we only consider three of seven nodes!* In the notation of Lemma 5.11, we have $\{x_1, y\} = \{3,1,2\} \subseteq V(G)$ and subgroup $H = \langle (4,5),\ (1,2),\ (3,6)(1,4)(2,5) \rangle$ of $Aut(G)$. Therefore, H contains all automorphisms of G that fix $\{3\}$ and maps 1 onto 2. Since this was the only non-trivial mapping that led to an automorphism, H contains all automorphisms.

Thus, we have found a permutation group H that generates $Aut(G)$ since it contains all automorphisms.

Second example

Figure 5.8 shows the branching tree when we first branch on the colour class of $\{1,2,4,5\}$. Note that, if we do that, the number of times that we need to make a branching decision changes, yet the number of leaves remains the same. And, again in contrast, the number of leaves that we cannot prune is larger than in our previous example. This indicates that making the "best" branching decisions can depend on both the graph and the implementation of different methods from the

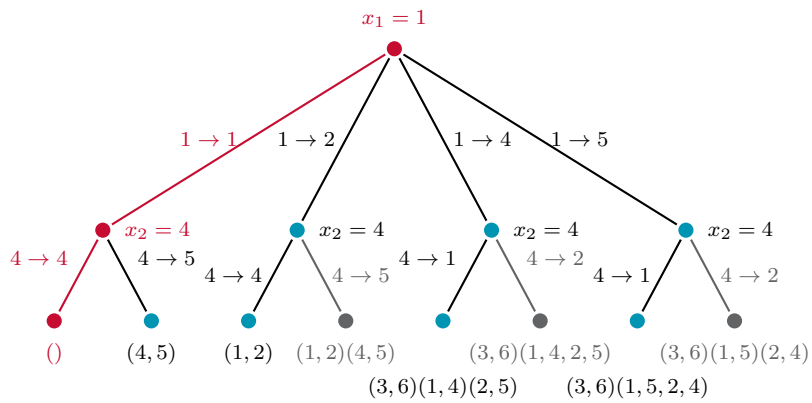


Figure 5.9: Pruned Branching Tree of Figure 5.5, branching on different colour classes.

algorithm. Note also that generally your branching decisions may depend on your previous decisions, in which case your branching tree is less symmetric than in these two examples.

Pseudocode

To concisely provide the algorithm in pseudocode, we need the following two definitions. The pseudocode is given in Algorithm 9.

Definition 5.12 (Trivial Mapping)**.** A mapping following (D, D) is called a trivial mapping.

Definition 5.13 (Trivial Node)**.** A node in the recursion tree that corresponds with a trivial mapping, is called a trivial node.

5.5 Order Computation and Membership Testing

This section explains how to compute the order of the group induced by a given generating set, as well as Membership testing. These two algorithms are necessary for Algorithms 7, 8, and 9 (as they are described). We also discuss recursive algorithms to solve these problems in polynomial time.

The algorithms rely on some knowledge about Orbits, Stabilisers and Transversals. Therefore we first discuss this. Then, we discuss the Order Computation and Membership Testing, accompanied by an example. Finally, we provide pseudocode for an algorithm to determine orbits and stabilisers.

Algorithm 9 UpdateGeneratingSet(D, I)

Input: D and I are equal length sequences of vertices of G
Action: Ensures that $\langle X \rangle$ contains all automorphisms f that follow D, I

ColourRefinement(G) with starting colouring $\alpha(D, I)$
if there is no automorphism following $\alpha(D, I)$ **then** Return
if there is unique automorphism f following $\alpha(D, I)$ **then**
  if $f \notin \langle X \rangle$ **then**
    $X := X \cup \{f\}$
  Return *to last visited trivial ancestor node*

Choose a branching cell C with $|C| \geq 2$
Choose a branching vertex $x \in C$
for $y \in C$, *start with $y = x$ when possible* **do**
  UpdateGeneratingSet$(D + x, I + y)$

5.5.1 Orbits, Stabilisers and Transversals

We first discuss orbits, stabilisers and transversals in the context of automorphisms groups. Let V be a set of vertices ($\alpha \in V$) and H be a permutation group on V.

Definition 5.14 (Orbit)**.** The orbit of vertex $\alpha \in V$ under H is the set of all vertices obtained by applying the permutations to α: $\alpha^H = \{p(\alpha) \mid p \in H\}$.

Definition 5.15 (Stabiliser)**.** The stabiliser of vertex $\alpha \in V$ under H is the subgroup of H of all permutations that leave α fixed: $H_\alpha = \{p \in H \mid p(\alpha) = \alpha\}$.

Definition 5.16 (Orbit transversal)**.** An orbit transversal of H modulo H_α is a set of permutations such that for each vertex β in α^H, it contains exactly one permutation $p : p(\alpha) = \beta$. In mathematical notation: $\bigcup_{\beta \in \alpha^H} u_\beta$, with $u_\beta \in H$ and $u_\beta(\alpha) = \beta$.

We note here that this definition of orbit transversal is specific to the stabiliser, yet there exists a more general definition of transversals for any subgroup H' of H. In fact, the definition given above is that of a left (or right, modulo our convention for multiplying permutations) transversal of H_α.

Definition 5.17 (Left transversal)**.** Let H' be a subgroup of the group H. The subset $S \subseteq H$ is a left transversal for H' in H if and only if every left coset of H' contains exactly one element of S.

In the following, when we write transversal (without any adjective), we refer to the orbit transversal.

Example

Consider again the cycle example (Figure 5.1). Here $V = \{0, 1, 2, 3, 4, 5\}$ and $H = \langle p, q \rangle$, where $p = (0\ 1\ 2)(4\ 5)$ and $q = (2\ 3)$. We investigate the orbit, stabiliser and a transversal for $\alpha = 0$.

The orbit of 0 contains all possible images of 0 under H. In this case, $0^H = \{1, 2, 3\}$, which is shown by the following permutations:

$$\begin{aligned} u_0 &= () \,, \\ u_1 &= p = (0\ 1\ 2)(4\ 5) \,, \\ u_2 &= p^{-1} = (0\ 2\ 1)(4\ 5) \,, \\ u_3 &= qp = (0\ 3\ 2\ 1)(4\ 5) \,. \end{aligned} \tag{5.3}$$

The stabiliser of 0 is the subgroup of permutations that leaves 0 untouched. Here $H_0 = \big\langle (2\ 3),\ (4\ 5),\ (1\ 3) \big\rangle$. These are indeed generated by p and q

$$q = (2\ 3), \quad p^3 = (4\ 5), \text{ and } p^{-1}qp = (1\ 3) \,.$$

A transversal of H modulo H_0 is a set of permutations such that for all elements in the orbit of 0 there is one permutation that maps it there. In this case, the u_0, u_1, u_2, and u_3 from 5.3 together are a transversal modulo H_0.

5.5.2 Order Computation

Proposition 5.18 (Orbit-Stabiliser Theorem)**.** *For any* $\alpha \in V$, $|H| = |H_\alpha| \cdot |\alpha^H|$

Proof. For any $p \in H_\alpha$ we have $p(\alpha) = \alpha$, therefore, the left cosets $u_\beta H_\alpha$ for all $\beta \in \alpha^H$ are distinct. By Lagrange's Theorem [3, Theorem 5.2.3], each of the cosets has equal order, so $|H| = |H_\alpha| \cdot |\alpha^H|$. □

Proposition 5.18 allows us to calculate $|H|$ recursively since it can be applied to $|H_\alpha|$ as well (choose a vertex with non-trivial orbit find its stabiliser and recurse). Important to note, is that choosing a vertex α with a trivial orbit will not help: $|H| = |H_\alpha| \cdot |\alpha^H| = |H| \cdot 1$. The recursion terminates when $|H_\alpha|$ only contains the trivial permutation since its order then is one.

Example (Continued)

Recall our previous example: $H = \langle p, q \rangle$, where $p = (0\ 1\ 2)(4\ 5)$ and $q = (2\ 3)$. Further more, $0^H = \{0, 1, 2, 3\}$ and $H_0 = \big\langle (2\ 3),\ (4\ 5),\ (1\ 3) \big\rangle$. Notice that 0 has a non-trivial orbit.

Recursively, using Proposition 5.18, we compute that $|H_0| = 12$. Therefore Proposition 5.18 gives: $|H| = |H_0| \cdot |0^H| = 12 \cdot 4 = 48$.

Implementation

To make Proposition 5.18 useful, two things need to be calculated efficiently, the orbit and the stabiliser subgroup. Calculating the orbit is not difficult and is discussed in Section 5.5.4. However, calculating the stabiliser subgroup is quite complex. Theorem 5.19 shows that it is possible in polynomial time, although we omit the proof from this reader. For the project, a Python program is provided that does this for you.

Theorem 5.19 (Generating a Stabiliser)**.** *Let $\{p_1, \ldots, p_k\}$ be a set of permutations on V, with $|V| = n$, let $H = \langle p_1, \ldots, p_k \rangle$, and $\alpha \in V$. In time polynomial in n and k we can compute a generating set Z for the stabiliser subgroup H_α, that contains at most n^2 permutations.*

Proof. Omitted. The interested reader can search for *Schreier's Lemma.* □

5.5.3 Membership Testing

Proposition 5.20 (Sifting)**.** *For all $\beta \in \alpha^H$, let $u_\beta \in H$ be a transversal element with $u_\beta(\alpha) = \beta$. For any permutation q with $q(\alpha) = \beta$,*

$$q \in H \iff u_\beta^{-1} q \in H_\alpha \,.$$

Proof. Let q be a permutation with $q(\alpha) = \beta$. Since $u_\beta \in H$, also $u_\beta^{-1} \in H$ and since $u_\beta(\alpha) = \beta$, we have $u_\beta^{-1}(\beta) = \alpha$. Suppose $q \in H$, then $u_\beta^{-1} q \in H$ and

$$u_\beta^{-1} q(\alpha) = u_\beta^{-1}(q(\alpha)) = u_\beta^{-1}(\beta) = \alpha$$

and $u_\beta^{-1} q \in H_\alpha$.

Now suppose $u_\beta^{-1} q \in H_\alpha$, then also $u_\beta^{-1} q \in H$ and $q = u_\beta u_\beta^{-1} q \in H$. □

Example (Continued)

Recall our previous example: $H = \langle p, q \rangle$, where $p = (0\ 1\ 2)(4\ 5)$ and $q = (2\ 3)$. Let us remind ourselves that $0^H = \{0, 1, 2, 3\}$ with the corresponding transversal

elements

$$\begin{aligned}
u_0 &= () \,, \\
u_1 &= p = (0\ 1\ 2)(4\ 5) \,, \\
u_2 &= p^{-1} = (0\ 2\ 1)(4\ 5) \,, \\
u_3 &= qp = (0\ 3\ 2\ 1)(4\ 5) \,.
\end{aligned}$$

Therefore, by Proposition 5.20, $f \in H$ if and only if $u_2^{-1} f = (0\ 1\ 2)(4\ 5) \circ (0\ 2) = (1\ 2)(4\ 5) \in H_0$. After this, we continue the recursion by finding a vertex that has a nontrivial orbit in H_0.

Computation Time

Let $X := \{p_1, \ldots, p_k\}$ be the set of permutations found so far, $H := \langle X \rangle$ the group generated by X, and f a new permutation found. To check if $f \in H$, we choose a vertex α with $|\alpha^H| \geq 2$. Now, either $f(\alpha) = \beta$ where $\beta \notin \alpha^H$, thus $f \notin H$. Or we use sifting: $f \in H$ if and only if $u_\beta^{-1} f \in H_\alpha$. Note that, by the Orbit-Stabiliser Theorem, $|H_\alpha| \cdot 2 \leq |H|$ and therefore we make progress.

Let n be the number of vertices of our graph. Then $|H| \leq n!$, since it is a permutation group. Since we always choose α with a nontrivial orbit, the recursive computation terminates in at most $\log_2 |H| \leq \log_2 n! \leq n \log_2 n$ iterations.

5.5.4 Orbits and Transversals

Theorem 5.19 shows that we can compute a stabiliser subgroup in polynomial time. Here, we introduce an algorithm that determines the order and a transversal of a vertex α. That it is correct and has polynomial running time is left as a simple exercise. The general idea is as follows.

The orbit is generated by first applying all permutations to α, and storing all possible images of α in a queue. Then we apply all permutations again to these images and add all new images to the queue as well. If we keep going till the queue is empty, we have generated the orbit of α. To also generate a transversal, each time we generate a new vertex in the orbit, we store the permutation. We end up with a set of permutations where each one maps α to a different β in its orbit.

Example

We again look at our example: $H = \langle p, q \rangle$ with $p = (0\ 1\ 2)(4\ 5)$ and $q = (2\ 3)$. We look for the orbit and transversal elements of $\alpha = 0$.

Algorithm 10 OrbitAndTransversal()

Input: Generating set $\langle p_1, \ldots, p_k \rangle$ of H, element $\alpha \in V$.
Output: Orbit α^H, and transversal elements u_β for all $\beta \in \alpha^H$.

```
u_α := ()  (The trivial permutation)
α^H := {α}
U := [α]  (a queue)

while U nonempty do
    β :=dequeue(U)
    for i ∈ {1, ..., k} do
        γ = p_i(β)
        if γ ∉ α^H then
            α^H := α^H ∪ {γ}
            enqueue γ to U
            u_γ := p_i u_β
```

We initialise with $0^H = \{0\}$ and $U = [0]$. Furthermore, we have that the trivial permutation yields 0. So $u_0 = ()$.

$0^H = \{0\}$	$U = [0]$	
Element:	Transversal:	
0	$()$	

Consider $0 \in U$: $p(0) = 1$ and $q(0) = 0$, only 1 is new, so we add 1 to the orbit and the queue. The transversal element that we add is $u_1 = p$.

$0^H = \{0, 1\}$	$U = [\emptyset, 1]$	
Element:	Transversal:	
0	$()$	
1	p	$= (0, 1, 2)(4, 5)$

Consider $1 \in U$: $p(1) = 2$ and $q(1) = 1$, only 2 is new so we add 2 to the orbit and the queue. The transversal element that we add is $u_2 = pu_1$.

$0^H = \{0, 1, 2\}$	$U = [\emptyset, \not{1}, 2]$	
Element:	Transversal:	
0	$()$	
1	p	$= (0\ 1\ 2)(4\ 5)$
2	pp	$= (0\ 2\ 1)$

Consider $2 \in U$: $p(2) = 0$ and $q(2) = 3$, only 3 is new, so we add 3 to the orbit and the queue. The transversal element that we add is $u_2 = qu_2$.

$0^H = \{0, 1, 2, 3\}$	$U = [\cancel{0}, \cancel{1}, \cancel{2}, 3]$	
Element:	Transversal:	
0	()	
1	p	$= (0\ 1\ 2)(4\ 5)$
2	pp	$= (0\ 2\ 1)$
3	qpp	$= (0\ 3\ 2\ 1)$

Consider $3 \in U$: $p(3) = 3$ and $q(3) = 2$, neither of them are new, so we add nothing to orbit or the queue.

$0^H = \{0, 1, 2, 3\}$	$U = [\cancel{0}, \cancel{1}, \cancel{2}, \cancel{3}]$	
Element:	Transversal:	
0	()	
1	p	$= (0, 1, 2)(4, 5)$
2	pp	$= (0, 2, 1)$
3	qpp	$= (0, 3, 2, 1)$

Now, the queue (U) is empty, so we terminate.

5.6 Implementation Details

By making use of the group properties of automorphisms of graphs, we can greatly speed up branching algorithms for counting automorphisms. In this chapter, we only discussed how to do this in those cases where we can say without knowing the graph that two branches of the branching tree result in automorphisms. However, the automorphisms that we find (or do not find) during branching can also guide us in which additional branches to prune. Doing so gains additional speed up, and is something that the reader can explore for themselves.

5.6.1 Points of Attention

When computing the order of the automorphism group from a given generating set, it is obvious that the computation times depend on the size of the set. However, correctness of the computation should not / cannot depend on the set being minimal (in the sense that no permutation can be removed from the set without

changing the group it generates). Thus, for correctness of the algorithm, membership testing is not needed. This leads us to the question why we add the additional step. The answer is that its use highly depends on the particular implementation of the remainder of the algorithms/methods. That is, a trade-off happens between the time necessary to test membership for each found automorphism and the speed up gained because of the reduced size of the generating set. It is up to the reader to decide which is favourable.

Chapter 6

Graph Structures and Preprocessing

6.1 Preprocessing

There are several things we might do before we start using our colour refinement algorithms on the graphs in order to decrease the computation time. This so-called preprocessing can range from simplifying graphs by handling twins in a clever way to detecting whether the graphs you are dealing with are trees, cycles or other graph classes. One of the simplest preprocessing you might already be applying is detecting whether the two graphs have the same number of vertices. The goal of preprocessing is to reduce the average computation time of the algorithm.

When preprocessing graphs, we need to be careful on how we do this. We want to simplify/remove the structure, but not remove any information as that might cause non-isomorphic graphs to become isomorphic. A possible solution for this could be: preprocessing labels that affect initial colouring and/or branching.

Note that preprocessing is always a trade-off. If you encounter a graph on which non of your preprocessing ideas work, the computational time that you spend on it is a delay before you can start your main algorithm.

In the other subsections of this chapter we discuss various of these graphs structures and per subsection we discuss how to detect the structure and how to use preprocessing labels to simplify the structure.

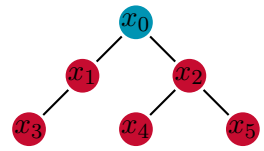


Figure 6.1: An example of a rooted tree where the blue vertex x_0 is chosen to be the root. x_1 and x_3 have a level of 1, x_2, x_4 and x_5 have a level of 2.

6.2 Trees

One of the preprocessing improvements we could implement is we first detect whether G and H are trees or forests and then apply a certain "new" algorithm. Because as it turns out, if G and H are trees or forests, we can actually find an algorithm that can solve both the GI-problem and the *#Aut problem* in $O(n)$. This is a great improvement compared to Algorithm 2. Recall the definition of a tree:

Definition 6.1 (Tree)**.** A tree is a connected acyclic undirected graph, meaning that any two vertices are connected by at most one path.

We can use the so-called AHU algorithm to solve both the isomorphism and *#Aut* problem, which will be explained in the following section.

Definition 6.2 (Rooted tree)**.** A **rooted** tree is a tree with a chosen vertex which is called the "root".

Definition 6.3 (Root)**.** A root is a chosen vertex x which has a level of 0, and all other vertices of that graph have a level relative to the root vertex, the distance between said vertex and x.

For an example of a rooted tree, consider Figure 6.1.

For rooted trees to be isomorphic we need a slight change in our definition of isomorphism as the root imposes an extra restriction on the bijection f which maps $V(G)$ to $V(H)$.

Definition 6.4. Two rooted trees G and H are isomorphic if the "non-rooted" versions of G and H are isomorphic, and the isomorphism f maps the root of G to the root of H.

Note that if two rooted trees are isomorphic, the "un-rooted" version of those trees are also isomorphic.

AHU algorithm

Now that we have the required definitions and knowledge we can start looking at the AHU algorithm. The AHU algorithm uses the labelling of vertices to give information about the structure of the rooted subtree with that vertex as the root. But now we will look at how to create such labels.

Let AHU(u) denote the label of vertex u determined from the AHU algorithm. We start at the leafs of the tree, if a vertex u is a leaf we will give it a label 0, AHU(u) = 0. If vertex u is *not* a leaf and has children $v_1, v_2, \cdots, v_k$, we give it label: AHU(u) = $\big(AHU(v_1), AHU(v_2), \cdots, AHU(v_k)\big)$. This is a recursive algorithm. However, note that we can change the order of different terms in the label of u any way we want, as we have not strictly defined what child of u is v_1 or v_2 etc. If we sort this in some way, we get the same label for the same vertices every time we run the algorithm, which is exactly what we want.

You can think of the rules of ordering these components yourself as this is not the most difficult task. We keep labelling the vertices based on the labels of their children until we eventually reach the root of the tree. Since we kept ordering the labels we now simply compare the label of the root of graph G and the root of graph H, if these are the same the two graphs are isomorphic. If the labels are not the same, there does not exist a bijection between these graphs.

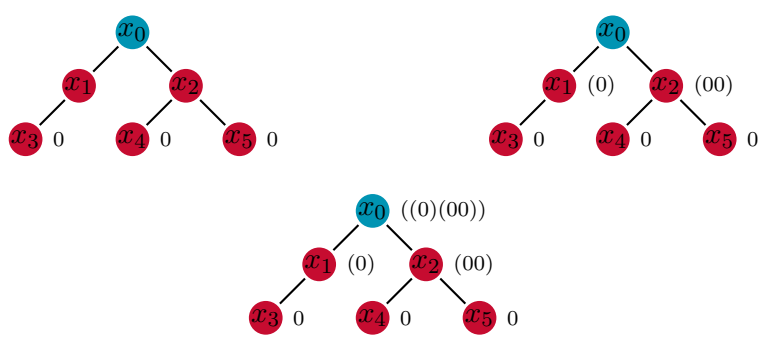


Figure 6.2: Example of AHU labelling on a tree.

As said before, this algorithm works for rooted trees, however we wish to solve the GI-problem and #Aut problem for un-rooted trees. It is now up to you to translate a un-rooted tree to a rooted tree, or possibly translate this algorithm to be able to work for un-rooted trees.

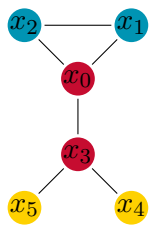


Figure 6.3: A graph containing both a set of twins and a set of false twins, note that this is some arbitrary colouring to distinct between the twins and false twins.

6.3 Twins

Definition 6.5 (True and False Twins)**.** Two vertices $u, v \in V(G)$ are twins if $(u, v) \in E(G)$ and $N(u) \backslash \{v\} = N(v) \backslash \{u\}$. They are false twins if $N(u) = N(v)$.

Consider the graph from Figure 6.3. When we look at vertices x_1 and x_2 they are both neighbours of each other and are neighbours of vertex x_0. So in this case, $x_1x_2 \in E(G)$ and $N(x_2) \backslash \{x_1\} = N(x_1) \backslash \{x_2\}$, which means that x_1 and x_2 are **twins**. If we now consider x_4 and x_5, they are not neighbours of each other, but have the same neighbourhood namely x_3. This means that $N(x_4) = N(x_5)$, hence x_4 and x_5 are **false twins**.

We can also have sets of k (false) twins, where each two vertices from the k vertices are (false) twins). To see how these structures can be problematic for a branching algorithm, consider replacing every vertex in a graph by a collection of k twins or false twins. Because colour refinement can not distinguish twins or false twins, the branching algorithm will have to branch on every twin, which increases the size of the branching tree with a factor $k!$ for every vertex (so $k!^n$ in total!). This is such a large factor, that traversing (a large part of) the branching tree quickly becomes infeasible.

Detecting false twins can be done in linear time by counting how many times each neighbourhood occurs. Detecting twins can also be done in linear time in a similar fashion, by comparing neighbourhoods that include the vertex itself. Before applying a branching algorithm, we can first detect (false) twins and change the graph or its colouring in such a way that the algorithm does not branch on the (false) twins. Note that just removing the twins (possibly except one) does not work, because graphs that were not isomorphic might become isomorphic this way (even when keeping an initial colouring!). One approach that can work is giving vertices that belong to the same set of twins different colours, but treating all similar sets of twins the same. Another approach is replacing sets of twins by a single vertex with the same neighbourhood, and giving that vertex a colour

corresponding to the set of twins it represents. When computing the number of automorphisms, we of course need to take into account that these changes affect the final answer by a factor $k!$ for each set of k twins.

6.4 Components

Theorem 6.6. *A **component** of a graph G is a connected subgraph that is not part of any larger connected subgraph. The components of any graph partition its vertices into disjoint sets.*

A graph with a lot of components can be interpreted as the disjoint union of smaller subgraphs called the components. We can detect the different components by executing a Breath First Search (BFS) on a one of the vertices of a graph G. If we keep track of all the vertices we visited during the BFS we will find all vertices that are "connected" to each other. This gives us one of the components of a graph. When this first BFS does not visit all vertices we can determine that there are multiple components in the graph. If there are multiple components in the graph we then repeat this BFS on one of the remaining vertices we have not visited in previous BFS iterations and we find another component. We keep doing these BFS on the vertices until we have visited all vertices.

We can use preprocessing labels to store information about for example; the number of vertices in the components or the colours of all the vertices in the components. This way we know that we do not have to branch on two vertices from different sized components or where the vertex colours of the components do not match. This reduces the graph isomorphism for big graphs to a graphs isomorphism problem for several smaller graphs, which in general is much quicker as it reduces the amount of possible branches.

6.5 Recursive Preprocessing

Recursively applying (different) preprocessing steps might also gain a lot! For example: a graph of 2 components where 1 component is a tree and the other component has a lot of twins, or twins that form new twins when removed. If we first preprocess for components and then for each component check for twins or trees we could greatly reduce the complexity of this problem.

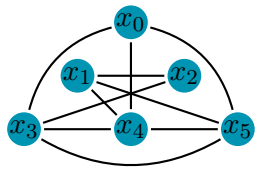


Figure 6.4: A graph that is not a tree, but does have a tree-like structure.

6.6 Other structures

Like the previous suggestions in this chapter, you can expand preprocessing of graphs for other structures as well. In general, algorithms that recognise particular structures can be fooled (easily) by adapting the underlying structure slightly. For example, if your algorithm recognises and preprocesses forests it will not recognise the graph in Fig. 6.4. However, when you look closely at that graph you may recognise that, in fact, it is the complement of the graph in Fig. 6.1. Since the complement of a graph G is the graph $\bar{G}$ such that

$$\{u, v\} \in E(G) \iff \{u, v\} \notin E(\bar{G}),$$

we have

Lemma 6.7. *Any isomorphism between two graphs G and H, is also an isomorphism between their complements $\bar{G}$ and $\bar{H}$.*

The proof of Lemma 6.7 is left to the reader.

Lemma 6.7 can be used algorithmically in several ways. One is to apply tree preprocessing to the graph in Fig. 6.4 by considering non-edges as edges and the other way around. Another way of achieving the same is by first taking the complement of the graph and then applying preprocessing. The latter approach can be helpful even if the complement of your graph does not have a specific structure. For example, if your algorithm is more efficient on graphs with fewer edges, one may assess if the original or the complement graph has fewer edges and apply the algorithm on that which has fewest. This can even be done component-wise.

Fig. 6.5 shows another example of a graph with a tree-like structure, that is not a tree. One may be tempted to think that this graph has the same number of automorphisms as the tree from Fig. 6.1, yet, it actually has more. This is because x_{13} and x_1, x_{24} and x_4, and x_{25} and x_5 are actually pairs of twins. In fact, running a recursive (false) twins preprocessing algorithm reduces this graph to a single vertex. However, we can again make a slight adjustment that ensures that even twin preprocessing does not help. Namely, by replacing the red vertices in

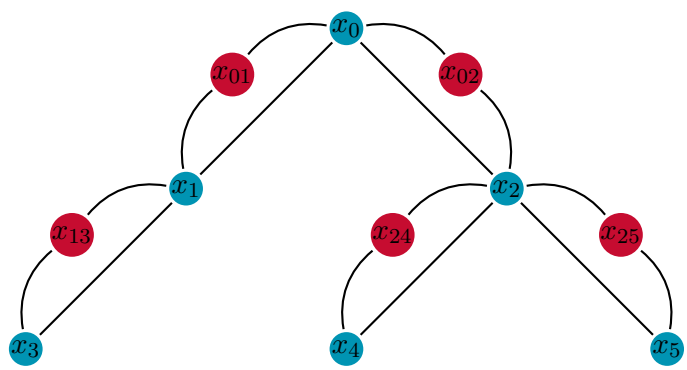


Figure 6.5: Another graph that is not a tree, but does have a tree-like structure.

Fig. 6.5 by cycles of length five (C_5 graphs) of which the first and the third vertex are respectively connected to either of the two blue neighbouring vertices. You can verify that the resulting graph has exactly the same number of automorphisms as the tree from Fig. 6.1. Preprocessing these types of structures is, of course, quite arbitrary (replace the C_5's by C_{2k+1}'s and the same structure is still present) and therefore not very useful.

Instead of preprocessing some arbitrary structure because of the issue highlighted by Fig. 6.5, one can think about what structures are particularly difficult for your algorithm to deal with but easily recognised, and try to preprocess those. Examples could be long paths or cycles as subgraphs. Several lists of named graphs can be found online, e.g., on Wikipedia: families of graphs and graph families. Always keep in mind that there is a trade-off between the time that preprocessing takes and the improvement that is obtained. Additionally, be aware that the example graphs that are provided in the project may not have (m)any graphs with the property that you want to preprocess. In that case, you need to come up with your own set of testing graphs. Teachers and TAs can help you when you want to create such graphs.

Bibliography

[1] J. Hopcroft. An n log n algorithm for minimizing states in a finite automaton. In Z. Kohavi and A. Paz, editors, *Theory of Machines and Computations*, pages 189–196. Academic Press, 1971.

[2] E. Klarreich. Graph Isomorphism Vanquished — Again, Jan. 2017.

[3] G. Loho and T. W. Judson. *Abstract Algebra for AM & TCS: A short introduction with applications*. 2023.

Index

#Aut problem *see* automorphism counting problem 3
GI-problem *see* graph isomorphism problem 2

automorphism, 3
  counting problem, 3, 21

bijection, 2
  colour-preserving, 7, 10
branching, 23
  rule, 36

colour refinement, 5
  fast, 37, 51
colouring, 6
  balanced, 16
  degree, 8
  discrete, 15
  stable, 15
  unbalanced, 16
  uniform, 7
component, 79

Deterministic Finite Automaton, 37
  minimisation, 38
DFA *see* Deterministic Finite Automaton 37

equivalence relation, 10

generating set, 57
graph isomorphism, 2
  problem, 1

isomorphic, 1, 2
isomorphism, 2

membership testing, 69

neighbourhood, 8
  identically-coloured, 10
  differently-coloured, 10

orbit, 67

partition, 18
  coarsest stable, 21
  into colour classes, 19
  refinement, 5, 19
  stable, 20, 40
permutation, 55
  group, 55
  multiplication of, 55
preprocessing, 75
  recursive, 79
pruning, 61

stabiliser, 67
symmetry, 1

transversal, 67
  left, 67
  orbit, 67
  right, 67
tree, 76
  rooted, 76
twins
  false, 78
  true, 78